



Departamentul Automatică și Informatică Industrială
Facultatea Automatică și Calculatoare
Universitatea POLITEHNICA din București



LUCRARE DE DISERTAȚIE

Dezvoltarea unei extensii Chrome pentru prevenirea atacului de tip phishing în navigarea web

Coordonator

Șl. Dr. Ing. Ana Magdalena Anghel

Absolvent

Ciuciu Anca-Maria

2025

Cuprins

1. Introducere	2
2. Prezentarea domeniului din care face parte lucrarea.....	3
2.1 Algoritmi de învățare automată	4
2.2 Rezultatele modelelor existente.....	7
2.3 Detectarea phishing-ului din aplicațiile web	10
3. Descrierea problemei abordate și a metodei de rezolvare propuse	12
3.1 Mediul de dezvoltare.....	12
3.2 Setul de date	13
4. Documentație tehnică	14
4.1 Caracteristicile unui URL.....	14
4.2 Procesarea setului de date	15
4.3 Modelul de învățare automată	16
4.4 Rezultate obținute și metricile modelului	19
5. Integrarea modelului de învățare automată.....	24
5.1 Conectarea extensiei la modelul de învățare automată	24
5.2 Testarea funcționalității integrate	26
5.3 Prezentarea aplicației și a funcționalităților	29
5.4 Raportarea URL-urilor	32
6. Concluzii și dezvoltări ulterioare	35
7. Bibliografie	41

1. Introducere

Atacul de tip phishing continuă să reprezinte o amenințare semnificativă pentru securitatea internetului, fiind una dintre cele mai utilizate modalități de obținere a datelor personale. Un studiu realizat de cei de la Intel a demonstrat ca 97% dintr-un grup de experti în Securitate cibernetică nu reușesc să identifice mail-urile ce conțin phishing. Motivul pentru care am ales să abordez acest subiect este în primul rând necesitatea de a crea un mediu online mai sigur și de a contracara amenințările cibernetice.

Atacurile cibernetice devin din ce în ce mai sofisticate pe măsura cu evoluția tehnologiei. Una dintre cele mai comune forme de phishing este prin intermediul URL-urilor unde atacatorii maschează URL-uri maligne ca fiind legitime, pentru a determina utilizatorii să le acceseze. Utilizatorilor li se poate cere, de asemenea, să introducă informații despre cardul de credit sau detalii despre contul bancar, precum și alte date sensibile. Odată ce aceste informații sunt colectate, atacatorii le pot folosi pentru a accesa conturile, date personale și identități, și pentru a descărca malware pe computerul utilizatorului.

Prin această lucrare se propune explorarea și dezvoltarea unei soluții pentru contracararea amenințărilor de tip phishing, prin implementarea unei extensii de browser bazată pe un model de învățare automată. Extensia de browser va avea ca scop detectarea și prevenirea acestui tip de atac cibernetic în timp real, furnizând un strat suplimentar de securitate în ceea ce privește navigarea pe internet. Extensia va monitoriza activitatea de navigare și va captura URL-urile accesate de către utilizator. Pentru fiecare URL capturat, extensia va extrage caracteristicile relevante necesare pentru detectarea phishingului. Aceste caracteristici pot include numărul de puncte, lungimea URL-ului, prezența unor cuvinte-cheie suspecte, etc. Extensia va utiliza modelul ML integrat pentru a clasifica URL-urile între legitime și potențial periculoase.

Pentru această etapă s-a început analizarea domeniului atât pentru alegerea unui set de date variat și reprezentativ pentru atacurile de phishing, cât și prin compararea performanței algoritmilor de învățare automată folosiți pentru această problemă. Totodată, pentru integrarea modelului de învățare automată, se va analiza și mediul de dezvoltare al extensiei de browser.

În continuare, se va explora structura lucrării, pornind de la setul de date și continuând cu algoritmi existenți, analiza rezultatelor și dezvoltările ulterioare propuse.

2. Prezentarea domeniului din care face parte lucrarea

Instrumentele anti-phishing pentru browser bazate pe învățarea automată reprezintă o abordare avansată și eficientă pentru detectarea și prevenirea atacurilor de phishing. Aceste instrumente utilizează algoritmi de învățare automată pentru a analiza și identifica caracteristicile suspecte ale URL-urilor și conținutului web, oferind protecție în timp real împotriva amenințărilor cibernetice.

Unul dintre instrumentele existente, dezvoltat pentru a combate atacurile cibernetice în cadrul website-urilor este Google Safe Browsing. [1] Safe Browsing folosește algoritmi de învățare automată pentru a analiza comportamentul site-urilor și pentru a detecta anomaliile care indică activități malițioase. Pe lângă modelele de învățare automată, Safe Browsing utilizează și reguli euristice pentru a identifica semnele comune ale site-urilor de phishing și malware. Aceasta include analiza codului sursă al paginilor web, căutarea de scripturi malițioase și verificarea dacă site-ul încearcă să exploateze vulnerabilități cunoscute. De asemenea, acesta se folosește și de tehnica de crowdsourcing. Google primește rapoarte de la milioane de utilizatori și dezvoltatori din întreaga lume, care contribuie la identificarea rapidă a noilor amenințări. Aceste rapoarte sunt analizate și integrate în bazele de date Safe Browsing pentru a îmbunătăți protecția oferită.

Harmony Search (HS) [2] este un algoritm inspirat din procesul muzical de căutare a armoniei perfecte. A fost dezvoltat de Zong Woo Geem în anul 2001 și este utilizat în diverse domenii de optimizare datorită simplității și eficienței sale. HS se bazează pe analogia dintre compunerea muzicală și soluționarea problemelor de optimizare. Acest algoritm poate fi utilizat pentru optimizarea selecției prin căutarea combinației optime de caracteristici care maximizează performanța clasificatorului.

Deep Packet Inspection (DPI) [3] este o tehnică de analiză a traficului de rețea care examinează în detaliu conținutul pachetelor de date. Spre deosebire de metodele tradiționale care analizează doar anteturile pachetelor, DPI inspectează atât anteturile, cât și payload-ul (conținutul efectiv) pentru a identifica, clasifica și bloca pachetele de date malițioase sau nedorite.

Modelul care înregistrează cea mai mare acuratețe este cel bazat pe rețelele neuronale convoluționale, un tip special de rețea neuronală artificială, concepută inițial pentru procesarea și recunoașterea imaginilor. CNN sunt alcătuite din mai multe straturi de neuroni, fiecare strat având rolul de a extrage caracteristici diferite din datele de intrare. Acestea sunt compuse din trei tipuri principale de straturi: straturi de convoluție, straturi de pooling și straturi complet conectate.

2.1 Algoritmi de învățare automată

Random Forest [4] este un model de clasificare care implica antrenarea mai multor arbori de decizie, iar predicția finală este determinată prin alegerea clasei majoritare dintre arbori. Pentru a asigura diversitatea între arbori, Random Forest utilizează conceptul de bagging, antrenând arbori pe seturi de date ușor diferite, dar cu aceeași dimensiune ca setul de antrenare inițial.

În cadrul fiecărui arbore de decizie, atunci când se face o decizie cu privire la ce caracteristică să fie folosită pentru a efectua o divizie, se face o selecție aleatoare a unui subset de caracteristici din care să se facă alegerea. Această diversificare este realizată prin selecția aleatorie a eșantioanelor, ceea ce poate duce la repetarea unor intrări. O altă metodă de diversificare constă în randomizarea feature-urilor utilizate, astfel încât arborii să nu adopte reguli identice în analiza setului de date.

Această diversificare este realizată prin selecția aleatorie a eșantioanelor, ceea ce poate duce la repetarea unor intrări. O altă metodă de diversificare constă în randomizarea feature-urilor utilizate, astfel încât arborii să nu adopte reguli identice în analiza setului de date.

Unul dintre beneficiile acestui algoritm este reducerea riscului de overfitting. Folosind mai mulți arbori antrenați pe subseturi diferite, Random Forest are o tendință mai mică de a se adapta prea bine la datele de antrenare.

Random Forest este utilizat în diverse domenii, cum ar fi recunoașterea de obiecte în imagini, analiza datelor, medicină, finanțe și multe altele, datorită capacității sale de a face predicții precise și de a gestiona o varietate de probleme de clasificare și regresie.

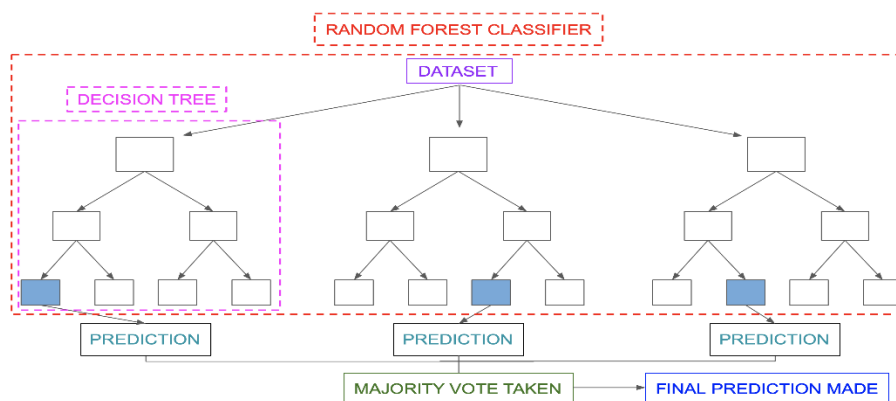


Fig. 1. Structura Random Forest [5]

K-Nearest Neighbors (KNN) [6] este un algoritm de învățare supervizată folosit pentru clasificare și regresie. Acesta se bazează pe ideea că obiectele similare sunt situate în apropiere în spațiul caracteristicilor.

Astfel, pentru a clasifica un nou punct de date, algoritmul KNN caută cei mai apropiați K vecini (bazați pe o metrică de distanță, de obicei distanța euclidiană) și decide clasa acestui

punct pe baza votului majoritar al vecinilor săi.

În cazul regresiei, valoarea atribuită unui punct nou este de obicei media sau mediana valorilor K vecini cei mai apropiați.

Pentru calcularea distanței Euclidiene, formula de bază între două puncte într-un spațiu n-dimensional este:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Unde x și y sunt două puncte, n este numărul de dimensiuni, iar x_i și y_i sunt valorile caracteristice pentru dimensiunea i.

Parametrul K reprezintă numărul de vecini considerat în procesul de clasificare sau regresie. O valoare mare a lui K poate duce la o decizie mai stabilă, dar poate ignora detalii locale, în timp ce o valoare mică a lui K poate duce la o sensibilitate crescută la zgomot și fluctuații ale datelor.

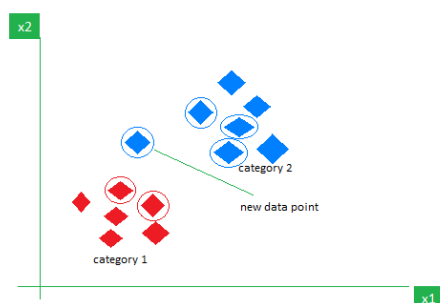


Fig. 2. Reprezentarea algoritmului KNN

În momentul în care se folosește acest algoritm, este importantă normalizarea datelor pentru a evita influențele nejustificate ale anumitor caracteristici asupra distanței calculate.

Algoritmul KNN este simplu de înțeles și implementat, fiind unul dintre primii pași în învățarea supervizată. Totuși, performanța sa poate fi sensibilă la dimensiunea datelor și trebuie să fie gestionată corespunzător pentru a evita overfitting-ul.

Arborii de decizie [7] sunt modele de învățare supervizată utilizate pentru clasificare și regresie. Acești arbori se bazează pe structuri de tip arbore, unde fiecare nod reprezintă o alegere între un set de opțiuni.

Un arbore de decizie începe cu un nod principal sau rădăcină, care reprezintă întreaga populație sau setul de date. Nodurile ulterioare se ramifică în funcție de caracteristicile datasetului. Aceste noduri reprezintă decizii sau întrebări bazate pe caracteristici, care împart

datasetul în subseturi mai mici. Fiecare frunză a arborelui este o clasificare sau o valoare pentru variabila țintă.

Într-un arbore de decizie, pentru a prezice clasa unui set de date dat, algoritmul începe de la nodul rădăcină al arborelui. Acest algoritm compară valorile atributului rădăcină cu atributul înregistrării (setul real de date) și, pe baza comparației, urmează ramura și sare la următorul nod. Pentru următorul nod, algoritmul compară din nou valoarea atributului cu celelalte sub-noduri și avansează mai departe. Procesul continuă până când ajunge la nodul frunză al arborelui.

Arborii de decizie sunt folosiți în mai multe domenii precum învățarea automată, analiza datelor și înțelegerea și luarea deciziilor bazate pe modele în diverse industrii, datorită naturii lor intuitive și adaptabilității într-o varietate de probleme de clasificare și regresie.

SVM (Support Vector Machine) [8] este un algoritm care vizează determinarea unui hiperplan cu marginea maximă pentru a separa două clase. Marginea reprezintă distanța dintre hiperplan și cel mai apropiat punct din fiecare clasă. Vectorii suport sunt puncte apropiate de hiperplan care influențează poziția și orientarea acestuia. Pentru clasificarea a mai multor clase, există două strategii: "one-vs-one", care implică utilizarea unui clasificator pentru fiecare pereche de clase ($N * (N - 1) / 2$ clasificatori, unde N este numărul de clase), și clasa predominant prezisă de model este considerată rezultatul final; și "one-vs-rest", care implică utilizarea unui clasificator pentru fiecare clasă, evaluând dacă o intrare aparține sau nu acelei clase în parte.

Gaussian Naive Bayes [9] este o variantă a algoritmului Naive Bayes, care face parte din familia de modele de învățare supervizată. Acest algoritm este adesea utilizat în probleme de clasificare și este specializat pentru date continue sau distribuții gaussiene (normale). Formula pentru teorema Bayes este:

$$P(A|B) = P(A) * P(B|A) / P(B)$$

- $P(A|B)$ este probabilitatea lui A dacă B s-a întâmplat
- $P(B|A)$ este probabilitatea lui B dacă A s-a întâmplat
- $P(A)$ este probabilitatea ca A să se întâmple
- $P(B)$ este probabilitatea ca B să se întâmple

Extreme Gradient Boosting (XGB) [10] este un algoritm puternic în învățarea automată, renumit pentru viteza, eficiența și precizia sa. Acesta construiește mai mulți arbori de decizie pentru a crea un model robust și precis. Prin utilizarea unor tehnici avansate, cum ar fi regularizarea și optimizarea, XGBoost împiedică overfitting-ul și îmbunătățește generalizarea modelului.

Fundamental, XGBoost construiește un model predictiv puternic agregând predicțiile mai multor învățători slabi, de obicei, arbori de decizie. Folosește o tehnică de boosting pentru a crea un model de ansamblu extrem de precis, având fiecare învățător slab după el corectând greșelile predecesorilor săi.

Metoda de optimizare (gradientul) minimizează o funcție de cost schimbând repetat

parametrii modelului în raport cu gradientii erorilor. Algoritmul prezintă, de asemenea, ideea de "gradient boosting cu arbori de decizie", în care funcția obiectiv este redusă prin calcularea importanței fiecărui arbore de decizie care este adăugat succesiv în ansamblu. Prin adăugarea unui termen de regularizare și utilizarea unui algoritm de optimizare mai avansat, XGBoost merge un pas mai departe și îmbunătățește precizia și eficiența.

Se folosește de "Sparsity Aware Split Finding" care este o tehnică specială pentru a trata eficient datele lipsă (sparse data) în timpul construcției arborilor de decizie, optimizează accesul la memorie astfel că se ține cont de ierarhia memoriei cache a computerului.

Astfel, se poate afirma faptul că robustețea algoritmului XGBoost este susținută de abordarea sa inovatoare în construcția arborilor, accesul conștient al cache-ului și tratarea datelor lipsă. Datorită adaptabilității, scalabilității și eficacității sale, algoritmul este o opțiune excelentă pentru o varietate de sarcini de învățare automată, în special cele care necesită un volum mare de date de antrenament.

2.2 Rezultatele modelelor existente

În cadrul acestei analize, am utilizat mai multe modele de învățare automată predefinite pentru a evalua performanța lor pe setul de date ales. Modelele selectate au fost: XGBoost, Decision Tree, Random Forest și KNN. Una din metricile analizate este acuratețea, ce reprezintă raportul între numărul de predicții corecte și numărul total de predicții făcute de model. Se poate observa și din graficul de mai jos faptul că, XGBoost înregistrează cea mai bună performanță în materie de acuratețe, 92.5%, cu un rezultat foarte apropiat de modelul Random Forest.

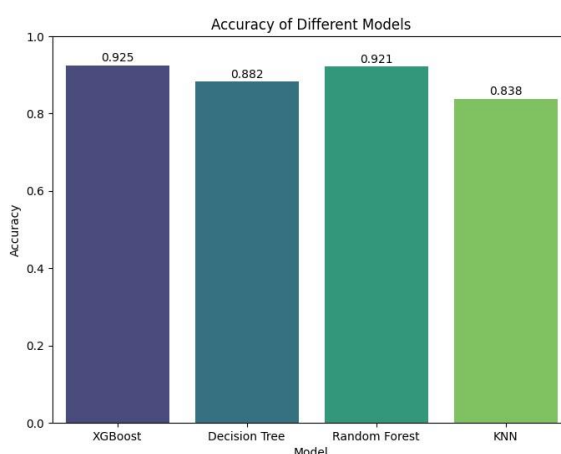


Fig. 3. Acuratețea modelelor existente

Totodată, se poate remarca faptul că cea mai mică performanță înregistrată este cea a modelului KNN cu o acuratețe de 83.8%.

Această diferență ar putea proveni din faptul că XGBoost poate captura interacțiuni complexe între caracteristici, în timp ce KNN se bazează pe similarități locale și poate să nu

fie la fel de eficient în identificarea unor structuri mai complexe în date.

De asemenea, în graficele de mai jos se pot observa alte două metrici, precizia și recall-ul pentru aceste modele. Precizia reprezintă în acest caz proporția exemplelor pozitive identificate corect de model din totalul exemplelor clasificate ca fiind pozitive, iar recall-ul este proporția exemplelor pozitive identificate corect de model din totalul exemplelor pozitive reale. Cu alte cuvinte, cât de bine identifică modelul toate exemplele pozitive reale.

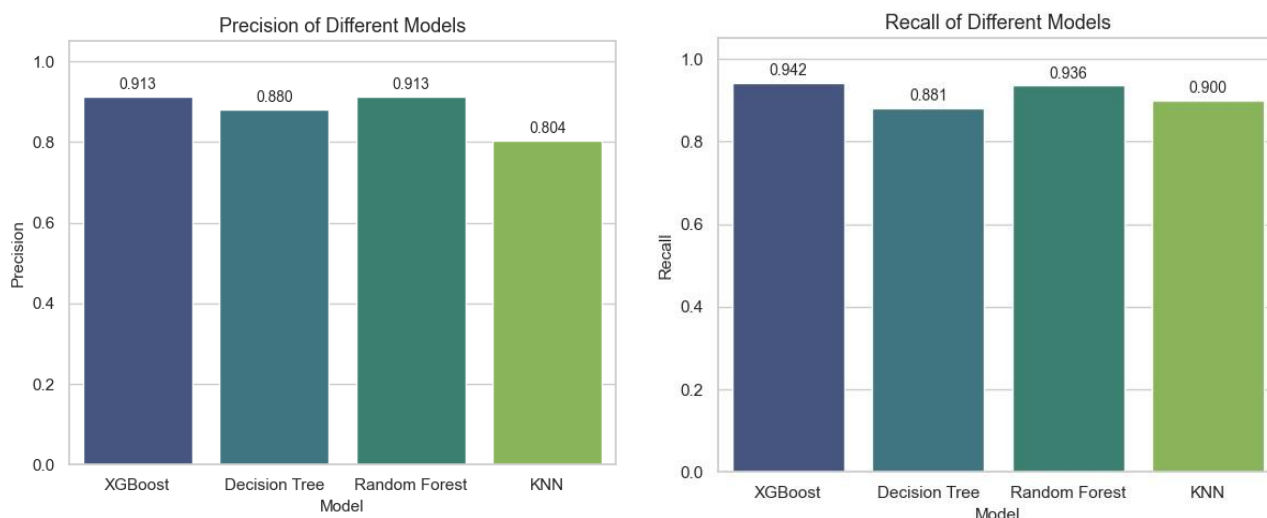


Fig. 4. Precizia și recall-ul modelelor existente

Integrarea unui model performant, precum XGBoost sau Random Forest, într-o extensie de browser aduce beneficii semnificative în prevenirea atacurilor de tip phishing în timp real. Datorită capacității lor ridicate de generalizare și a performanțelor bune în identificarea corectă a linkurilor periculoase, aceste modele pot evalua rapid URL-urile accesate de utilizator și pot bloca sau avertiza asupra celor suspecte înainte ca utilizatorul să interacționeze cu ele.

Astfel, se reduce riscul compromiterii datelor personale sau financiare, iar experiența de navigare rămâne fluentă, fără întârzieri semnificative. Mai mult, aceste modele pot fi actualizate incremental, pe baza unor exemple noi de atacuri, adaptându-se continuu la tacticile tot mai sofisticate ale atacatorilor.

Scorul F1 este o metrică de evaluare utilizată frecvent în problemele de clasificare, mai ales atunci când datele sunt dezechilibrate sau când este important atât să se evite alarmele false (precizie), cât și să nu se rateze cazuri reale (recall). Mai exact, scorul F1 reprezintă media armonică dintre precizie și recall.

Această metrică oferă o imagine de ansamblu echilibrată asupra performanței unui model, penalizând modelele care au o diferență mare între precizie și recall. În graficul de mai jos, se observă că modelul **XGBoost** obține cel mai mare scor F1 (0.927), ceea ce

confirmă performanța sa superioară în detectarea linkurilor de tip phishing, deja indicată și de precizie și recall. **Random Forest** urmează îndeaproape cu un scor de 0.924, ceea ce îl face o alternativă viabilă. În schimb, **KNN** are cel mai mic scor F1 (0.849), sugerând o performanță mai slabă în echilibrarea între precizie și recall.

Astfel, scorul F1 întărește concluzia că XGBoost este cel mai potrivit model pentru a fi integrat într-un sistem anti-phishing, oferind un compromis optim între acuratețea detecției și rata de acoperire a atacurilor reale.

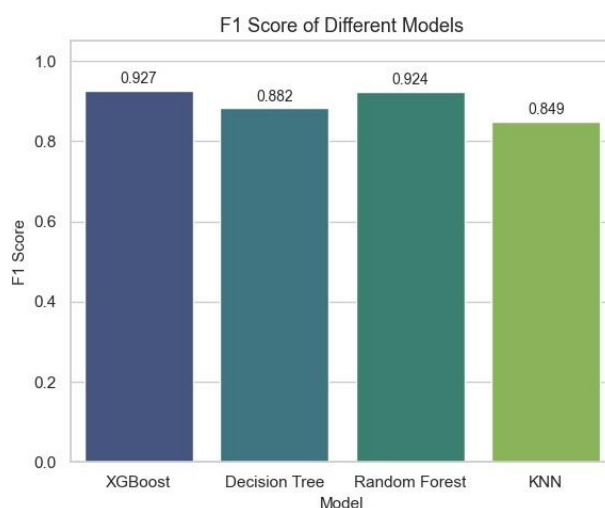


Fig. 5. Scorul F1 al modelelor existente

Pe baza analizei metricilor de evaluare — precizie, recall și scorul F1 — se poate concluziona că modelul **XGBoost** oferă cele mai bune performanțe în clasificarea linkurilor de tip phishing. Acesta obține cea mai mare acuratețe (0.925), ce mai mare precizie (0.913), cel mai mare recall (0.942) și cel mai ridicat scor F1 (0.927), demonstrând astfel o capacitate excelentă atât de a identifica corect linkurile periculoase, cât și de a minimiza alertele false. **Random Forest** se clasează foarte aproape de XGBoost, fiind de asemenea un candidat viabil, însă ușor mai slab pe toate cele trei metrici.

În schimb, modelele **Decision Tree** și **KNN** prezintă performanțe mai scăzute, în special în ceea ce privește scorul F1, ceea ce le face mai puțin potrivite pentru un sistem de detecție în timp real.

Prin urmare, pentru o implementare eficientă într-o extensie de browser destinată protejării utilizatorilor împotriva phishingului, **XGBoost** reprezintă cea mai robustă alegere, echilibrând cu succes detecția corectă a amenințărilor cu reducerea alarmelor false.

2.3 Detectarea phishing-ului din aplicațiile web

Browser-ele web sunt platforme care gestionează fluxul de informații, de la încărcările și descărcările de pe internet până la vizitarea site-urilor web. O extensie reprezintă un mic segment de software ce poate fi adăugat pentru a personaliza aspectul sau funcționalitatea browserului. Ele pot proveni de la dezvoltatorii browserelor sau de la terți ce doresc să aducă noi facilități pe care browserul actual nu le furnizează.

Extensiile de browser sunt capabile să ofere o gamă variată de funcționalități, de la criptarea emailurilor, blocarea reclamelor și stocarea rapidă a parolelor, până la corectarea automată a greșelilor de scriere. Aceste extensii acționează ca agenți specializați, influențând maniera în care informațiile circulă prin browser.

Cu toate acestea, pentru a funcționa în mod corespunzător, extensiile solicită adesea permisiuni extinse asupra browserului. Unele pot necesita acces la aproape toate detaliile pe care browserul le percepe, inclusiv site-urile vizitate, tastatura utilizată sau chiar parolele stocate.

Pentru adăugarea și testarea extensiei de browser se va folosi Developer mode ce permite încărcarea fișierului ce are la bază implementarea funcționalității. Totodată, aceasta va fi disponibil și în lista de extensii puse la dispoziție de browser și va putea fi dezactivată sau configurată în funcție de necesitățile utilizatorului.

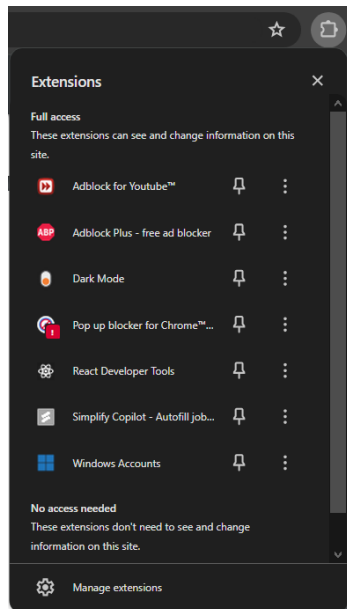


Fig. 6. Vizualizarea extensiilor web din browser

Browser-ul vizat pentru a integra modelul de învățare automată ce va depista potențialul atac de tip phishing este Chrome. Așadar, prin intermediul documentației pusă la dispoziție, Chrome for Developers, se va dezvolta un modul cu ajutorul limbajelor HTML, CSS și Javascript ce va avea ca scop avertizarea utilizatorilor cu privire la eventualitate accesării unor

pagini web nesigure. În figura de mai jos, este un exemplu de astfel de extensie ce afișează link-ul paginii curente și evaluarea acestuia.

O altă metoda de a depista atacul de tip phishing prin intermediul unei extensii web, în afară de cea a analizei URL-ului ar fi analiza complexă a codului HTML/CSS/Javascript. Astfel se poate face o analiză a surselor externe încărcate de pagină, cum ar fi fișierele JavaScript sau CSS, pentru a verifica dacă acestea sunt provenite de la surse de încredere. Unele atacuri de phishing încearcă să încarce resurse de la servere neautorizate sau suspecte.

Analizarea codului HTML pentru redirectionări neașteptate, legături care par să conducă către alte domenii decât cele indicate, sau modificări în comportamentul butoanelor și linkurilor poate fi un semn al unei posibile tentative de phishing.

Pentru a naviga și analiza structura codului HTML în mod eficient, se poate folosi biblioteca BeautifulSoup din Python, aceasta fiind specializată în parsarea și manipularea documentelor HTML, facilitând identificarea și extragerea de informații specifice sau a elementelor relevante dintr-o pagină web.

O altă modalitate de accesare a paginilor ce conțin atacuri de tip phishing ar fi prin intermediul mail-urilor cu același scop. Acestea în general conțin atașate link-uri către pagini web suspecte, cu intenții frauduloase.

Primul și poate cel mai important aspect în detectarea phishing-ului în e-mailuri este verificarea atentă a sursei. Acest lucru implică analizarea adresei de e-mail a expeditorului, fiind atenți la eventualele diferențe subtile care pot indica o adresă falsă sau frauduloasă. Adesea, atacatorii încearcă să creeze adrese care par a fi legitime, dar au mici variații, deci o atenție deosebită la detalii este esențială.

Un alt aspect crucial este limbajul și tonul folosit în e-mailuri. E-mailurile de phishing pot folosi limbaj presant sau alarmant, încercând să creeze o reacție rapidă și impulsivă din partea destinatarului. Greșelile gramaticale sau ortografice pot fi, de asemenea, indicii ale unui e-mail suspect. În plus, atașamentele și linkurile din e-mailuri trebuie tratate cu prudență. Evitarea clicului pe linkuri necunoscute sau descărcarea de fișiere din e-mailuri suspecte poate preveni accesul la malware sau site-uri web periculoase.

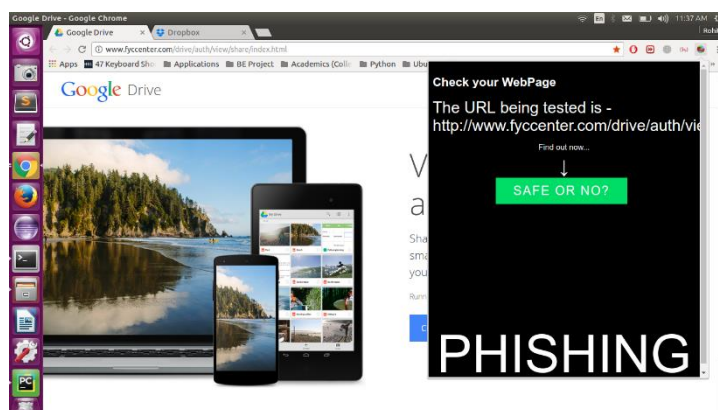


Fig. 7. Exemplu de extensie de browser pentru detectarea phishing-ului

Astfel, în imaginea de mai sus, se poate observa un model de extensie de browser ce analizează în mod automat URL-ul paginii web accesate și oferă posibilitatea utilizatorului de a verifica dacă acea pagină conține un potențial atac de tip phishing.

3. Descrierea problemei abordate și a metodei de rezolvare propuse

3.1 Mediul de dezvoltare

Pentru dezvoltarea modelului de deep learning se va folosi Python 3.9.13, împreună cu următoarele biblioteci:

- pandas și numpy - prelucrarea datelor
- sklearn - încărcarea modelelor de învățare automată
- keras și tensorflow - Deep Learning
- pickle - salvarea setului de vectorizări
- matplotlib - realizarea graficelor

Tehnica folosită pentru antrenarea rețelei neurale ce va detecta link-urile suspicioase este Deep Learning. Deep Learning [11] este o tehnică de învățare automată care învață caracteristici direct din date. Datele pot fi imagini, text sau sunet. Aceasta necesită o cantitate mare de date etichetate și permite Unității de Procesare Grafică (GPU) să antreneze rețele profunde într-un timp mai scurt.

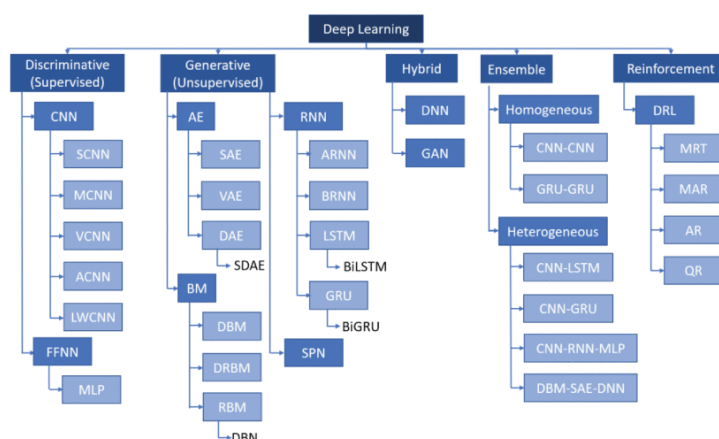


Fig. 8. Clasificarea rețelelor neurale [12]

Pentru antrenarea propriu zisă a modelului de clasificare s-a folosit o rețea neurală de tip *Feedforward* (Sequential), din biblioteca Keras. Feedforward Neural Network (FFNN) se referă la o arhitectură în care informația se deplasează secvențial, într-o singură direcție, de la stratul de intrare, la cel de ieșire fără a exista bucle în rețea. În figura de mai jos se poate observa clasificarea rețelelor aparținând domeniului Deep Learning.

Termenul "supervised" se referă la modul în care rețeaua este antrenată. Într-un context supervizat, rețeaua este învățată cu ajutorul unui set de date etichetate. Adică, pentru fiecare intrare din setul de date, se furnizează și o etichetă corespunzătoare care indică clasa sau categoria corectă. Antrenarea presupune ajustarea parametrilor rețelei astfel încât să minimizeze diferența dintre ieșirile prezise și etichetele reale.

3.2 Setul de date

Setul de date [13] ce va fi folosit pentru antrenarea modelului este preluat de pe Kaggle din care au fost preluate 48 de caracteristici extrase din 5000 de pagini web de phishing și 5000 de pagini web legitime, descărcate în perioada ianuarie-mai 2015 și mai-iunie 2017. Aceste caracteristici reprezintă niște valori numerice calculate pe baza URL-ului. Uniform Resource Locator (URL) este creat pentru a identifica pagini web. Figura de mai jos [14] arată caracteristicile principale din structura unui URL tipic.

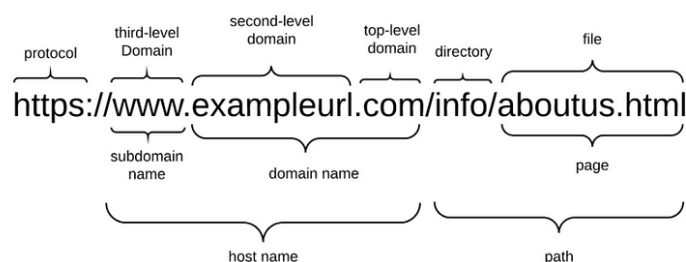


Fig. 9. Caracteristicile principale din structura URL

Caracteristicile analizate în contextul acestui studiu referitor la phishing includ aspecte precum nivelul de subdomenii, lungimea URL-ului, numărul de cratime, simboluri "@" și "~", utilizarea protocolului HTTPS, prezența adresei IP, etc.

Pentru a realiza o analiză mai concisă și ușor de urmărit, s-a optat pentru eliminarea a 26 de coloane din setul de date din totalul de 50. Această decizie a fost luată în considerare pentru a păstra doar acele coloane care au avut un impact semnificativ asupra rezultatelor analizei, facilitând astfel înțelegerea și interpretarea rezultatelor.

De asemenea, această decizie a fost luată din cauza faptului că, în momentul în care vor fi testate manual link-urile, caracteristicile vor fi extrase folosind expresii regulate.

4. Documentație tehnică

4.1 Caracteristicile unui URL

În contextul detectării URL-urilor de phishing, acest proces include colectarea, pregătirea și standardizarea datelor. Colectarea unui set de date cuprinzător de URL-uri este esențială pentru antrenarea modelelor de detectare precise. Asigurarea calității și integrității datelor prin eliminarea duplicatelor și verificarea autenticității URL-urilor îmbunătățește fiabilitatea analizei. În plus, analizarea și standardizarea URL-urilor într-un format consistent permite extragerea semnificativă a caracteristicilor. [15]

Aceste caracteristici servesc drept intrări pentru algoritmi de învățare automată, ajutând la clasificarea precisă a URL-urilor ca fiind cu potențial malițios sau nu. Diverse tipuri de caracteristici pot fi extrase din URL-urile de phishing, inclusiv caracteristici bazate pe domeniu, pe cale, pe gazdă și pe conținut.

Caracteristicile bazate pe domeniu se concentrează pe proprietățile domeniului URL-ului. Astfel, prin intermediul unui script de python de feature extraction s-a definit o serie de funcții pentru extragerea caracteristicilor dintr-un URL, care sunt utile pentru detectarea URL-urilor de phishing. Acestea sunt câteva dintre funcțiile implementate pentru extragerea caracteristicilor:

Funcții de numărare a simbolurilor:

- `count_dashes_in_hostname(url)`: numără cratimele din numele gazdei
- `count_at_symbol(url)`: numără simbolurile "@" din URL
- `count_tilde_symbol(url)`: numără simbolurile "~" din URL
- `count_percent_symbol(url)`: numără simbolurile "%" din URL
- `count_ampersand_symbol(url)`: numără simbolurile "&" din URL

Funcții pentru caracteristici de nivel de cale și subdomeniu:

- `get_subdomain_level(url)`: returnează nivelul subdomeniului (numărul de subdomenii)
- `get_path_level(url)`: returnează nivelul căii (numărul de componente ale căii)

Funcții pentru lungimi:

- `get_url_length(url)`: returnează lungimea URL-ului
- `count_path_length(url)`: returnează lungimea căii
- `count_query_length(url)`: returnează lungimea componentei de interogare

Funcții pentru verificarea anumitor condiții:

- `check_https(url)`: verifică dacă URL-ul folosește HTTPS
- `check_ip_address(url)`: verifică dacă URL-ul este o adresă IP
- `check_double_slash_in_paths(url)`: verifică dacă există dublu slash în cale

- `check_ext_favicon(url)`: verifică dacă URL-ul are un favicon extern

Funcții pentru componente de interogare:

- `count_query_components(url)`: numără componentele de interogare din URL

În final, caracteristicile vor fi extrase prin intermediul funcției de *feature_extraction(url)*, ce utilizează toate funcțiile de mai sus și nu numai, pentru a extrage caracteristicile din URL și pentru a returna o listă de valori ale caracteristicilor.

Odată ce caracteristicile sunt extrase, pasul următor implică selecția și reducerea dimensionalității caracteristicilor. Selecția caracteristicilor ajută la identificarea celor mai relevante și discriminative caracteristici, reducând complexitatea computațională și îmbunătățind performanța modelului. Tehnicile de reducere a dimensionalității reduc în continuare numărul de caracteristici, păstrând în același timp informațiile esențiale, facilitând analiza și vizualizarea eficientă a datelor.

4.2 Procesarea setului de date

În cadrul unui model de învățare automată, caracteristicile specifice ale URL-urilor sunt utilizate pentru a antrena algoritmi să recunoască tipare și anomalii asociate cu atacurile de phishing. Inițial, URL-urile sunt colectate și preprocesate pentru a extrage trăsăturile relevante, cum ar fi lungimea URL-ului, numărul de subdomenii, utilizarea caracterelor speciale și entropia. Aceste trăsături sunt apoi transformate în vectori de caracteristici care servesc drept intrări pentru algoritmi de învățare automată, cum ar fi SVM, arborii de decizie sau rețelele neuronale. Algoritmii sunt antrenați pe seturi mari de date, învățând să diferențieze URL-urile legitime de cele de phishing pe baza acestor trăsături. După antrenament, modelul poate evalua URL-uri noi, clasificându-le ca legitime sau de phishing în funcție de similaritatea lor cu tiparele învățate, asigurând astfel o detecție precisă și eficientă a potențialelor amenințări.

Entropia este o măsură a incertitudinii sau a dezordinii într-un set de date. În contextul URL-urilor, entropia cuantifică complexitatea și variabilitatea caracterelor dintr-un URL. Cu cât un URL este mai dezordonat și mai aleatoriu, cu atât entropia acestuia va fi mai mare. Entropia unui URL poate fi calculată folosind formula entropiei Shannon, care este utilizată pentru a măsura diversitatea informației într-o secvență de caractere și presupune calcularea frecvenței caracterelor, a logaritmului probabilității, realizându-se suma ponderată. Astfel, formula pentru calcularea entropiei H [16] a unui URL este:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

unde:

- X reprezintă setul de caractere din URL

- $p(x_i)$ este probabilitatea de apariție a caracterului x_i

Un posibil instrument de analiză a acestui set de date ar fi matricea de corelație, oferind o imagine de ansamblu asupra relațiilor dintre variabile. În contextul detectării atacurilor de tip phishing, aceasta ajută la identificarea caracteristicilor relevante și la reducerea redundanței, optimizând astfel modelele de învățare automată utilizate în detecție.

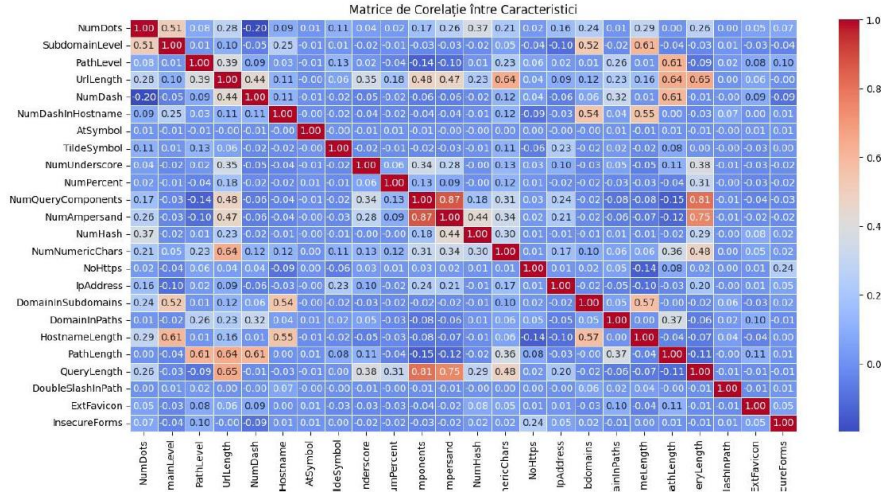


Fig. 10. Matricea de corelație între caracteristici

4.3 Modelul de învățare automată

Rețelele neurale sunt modele de învățare automată, inspirate de structura și funcționarea creierului uman, compuse din straturi de neuroni interconectați, informația fiind transmisă prin conexiunile ponderate între aceștia.

În cadrul rețelelor neuronale, un neuron este un model matematic, o unitate de procesare a informației, ce calculează media ponderată a intrărilor sale și aplică un decalaj asupra rezultatului. Decalajul este o valoare constantă adăugată la rezultatul ponderat înainte de a fi trecut prin funcția de activare. Acesta permite modelului să fie mai flexibil și să se adapteze mai bine la datele de intrare.

Rezultatul ponderat al intrărilor peste care a fost aplicat decalajul este trecut mai apoi printr-o funcție de activare non-lineară. Funcțiile de activare folosite cel mai des sunt funcția sigmoidă, funcția ReLU (Rectified Linear Unit) și funcția tanh. Aceste funcții transformă suma ponderată a intrărilor unui neuron într-o valoare de ieșire într-un anumit interval. Această ieșire poate fi folosită ca intrare pentru alți neuroni sau poate fi chiar rezultatul final al rețelei. Această arhitectură este dispusă pe trei straturi.

Stratul de intrare reprezintă primul strat dintr-o rețea neurală tipică. În această zonă, neuronii primesc informațiile de intrare, acestea fiind procesate de funcția de activare și o transmit către stratul următor, în funcție de o valoare de prag definită. La final, rezultatul este

scalat utilizând ponderile prestabilite, care sunt asociate cu conexiunile dintre neuronii din stratul de ieșire și neuronii din cel de intrare.

O rețea neurală poate avea unul sau mai multe straturi ascunse, astfel că, neuronii din acest strat primesc intrările fie de la neuronii din stratul de intrare, fie de la neuronii din stratul ascuns anterior. Aici se prelucrează intrarea prin funcția de activare neliniară și se propagă rezultatul către neuronii din următorul strat.

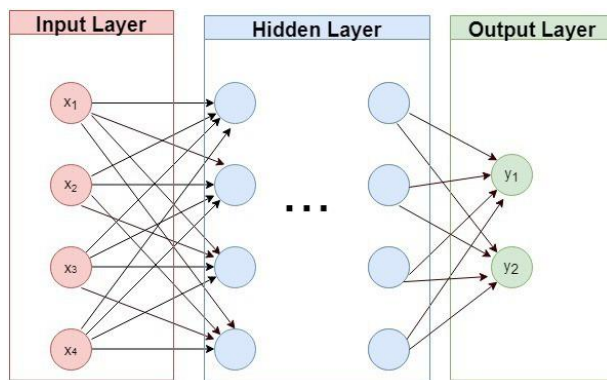


Fig. 11. Schema unei rețele neuronale artificiale

Stratul de ieșire este ultimul strat, care marchează finalul logic al rețelei. Aici, neuronii din stratul de ieșire primesc intrările de la straturile anterioare, procesându-le tot prin funcții de activare specifice. Rezultatul acestui strat poate fi o clasificare binară, ce conține doar un neuron, indicând apartenența la una din cele două clase, o valoare de predicție, sau precum este și în cazul aplicației dezvoltate, poate fi o clasificare cu mai multe clase.

Tehnica folosită pentru antrenarea rețelei neuronale ce va detecta link-urile suspicioase este Deep Learning. Deep Learning este o tehnică de învățare automată care învață caracteristici direct din date. Datele pot fi imagini, text sau sunet. Aceasta necesită o cantitate mare de date etichetate și permite Unității de Procesare Grafică (GPU) să antreneze rețele profunde într-un timp mai scurt.

Pentru antrenarea propriu zisă a modelului de clasificare s-a folosit o rețea neurală de tip Feedforward (Sequential), din biblioteca Keras. Feedforward Neural Network (FFNN) se referă la o arhitectură în care informația se deplasează secvențial, într-o singură direcție, de la stratul de intrare, la cel de ieșire fără a exista bucle în rețea.

În această lucrare a fost abordată problema clasificării binare utilizând rețele neuronale artificiale. Pentru dezvoltarea și antrenarea modelului de clasificare s-a folosit o rețea neuronală artificială de tip feedforward (Sequential) implementată prin intermediul bibliotecii Keras. Arhitectura feedforward implică propagarea informației într-o singură direcție, de la stratul de intrare către stratul de ieșire, fără existența unor bucle interne.

Setul de date utilizat a fost împărțit în date de antrenare, validare și testare prin intermediul funcției `train_test_split()`, folosind raportul 70:10:20. Astfel, 70% din date au fost

folosite pentru antrenare, 10% pentru validare și optimizarea hiperparametrilor, iar 20% pentru evaluarea finală a performanței modelului.

Pentru determinarea configurației optime de hiperparametri a fost utilizată tehnica Grid Search, o metodă automată și sistematică de explorare a spațiului de hiperparametri. Această metodă implică generarea tuturor combinațiilor posibile de valori predefinite și evaluarea performanței modelului pentru fiecare dintre acestea. Configurația care obține cele mai bune rezultate, conform metricilor de evaluare, este selectată drept configurație optimă. Prin această abordare, procesul de identificare a parametrilor optimi este eficientizat și se asigură performanța maximă a modelului în raport cu datele disponibile.

Printre hiperparametrii investigați se numără numărul de neuroni din cele trei straturi ale rețelei neuronale. Pentru primul strat s-au testat valori de 1024 de neuroni, în timp ce pentru următoarele două straturi valorile au fost de 512 neuroni fiecare. În general, creșterea numărului de neuroni poate spori capacitatea de învățare a modelului, dar poate genera și riscul de supra-antrenare (overfitting) și creșterea timpului de antrenare.

Pentru a limita riscul supra-antrenării, în arhitectura modelului au fost adăugate două straturi de tip dropout, poziționate după fiecare strat ascuns (straturile 2 și 3). Valorile testate pentru dropout au fost de 0.1 și 0.3. De exemplu, o valoare a dropout-ului de 0.1 determină dezactivarea temporară, aleatorie, a aproximativ 10% din neuroni în fiecare etapă de antrenare, fapt ce ajută la obținerea unei reprezentări robuste și generalizabile a datelor.

Alți doi hiperparametri importanți adăugați în analiza Grid Search au fost algoritmul de optimizare și rata de învățare. Algoritmii de optimizare evaluați au fost Stochastic Gradient Descent (SGD), o metodă clasică ce actualizează parametrii pe baza gradientului calculat pe subgrupuri ale datelor de antrenare, și Adaptive Moment Estimation (Adam), o metodă mai avansată care folosește estimări adaptative ale momentelor gradientului. De asemenea, au fost analizate două valori pentru rata de învățare, 0.001 și 0.01, aceasta reprezentând viteza cu care modelul ajustează parametrii în fiecare etapă de antrenare. O rată mai mică duce la convergență mai lentă, dar mai precisă, în timp ce o rată mai mare permite convergența rapidă, dar poate cauza instabilitate în procesul de învățare.

În urma realizării produsului cartezian între toți acești hiperparametri, Grid Search generează fiecare combinație posibilă și o evaluează prin construirea și antrenarea unui model pentru fiecare configurație. Modelul este construit prin metoda Sequential, care permite adăugarea straturilor neuronale într-o manieră ordonată. Ultimul strat, stratul de ieșire, conține un singur neuron cu funcția de activare sigmoid, oferind probabilitatea ca URL-ul analizat să aparțină clasei phishing.

În continuare, se va compila acest model specificându-se o serie de parametri. Primul parametru este tipul funcției de pierdere. În acest caz, s-a folosit `binary_crossentropy`, aceasta fiind adesea utilizată în problemele de clasificare binară, pentru a măsura discrepanța între distribuția de probabilități prezise de model și distribuția reală a claselor. Unul dintre obiective este acela de a minimiza valoarea funcției de pierdere, adică, găsirea setului optim de ponderi și parametri care să genereze cele mai bune predicții.

Pentru salvarea ponderilor asociate celei mai bune epoci s-a folosit un obiect de tipul `ModelCheckpoint`, realizându-se un callback. Antrenarea se pornește cu funcția `fit()`, ce utilizează datele de antrenare și validare. Pentru acest proces s-au folosit 512 de exemple de antrenare care sunt procesate într-o iterație și 100 de epoci. Rezultatul căutării celui mai bun set de hiperparametri este reprezentat de configurația: `strat1 = 512` neuroni; `strat2 = 1024` neuroni; `strat3 = 512` neuroni; `dropout1 = 0.1`; `dropout2 = 0.3`; clasa de optimizare = 'adam'; rata de învățare = 0.01. Acuratețea finală a modelului este de 91.5%. Această configurație va fi în continuare folosită pentru antrenarea propriu-zisă a modelului.

4.4 Rezultate obținute și metricile modelului

Unul dintre indicatorii ce ar putea să ne confirme antrenarea corespunzătoare a modelului este reprezentarea funcției de pierdere. Scopul principal al funcției de pierdere (**loss function**) este să cuantifice eroarea modelului în timpul antrenării, astfel încât să poată fi folosită pentru a ajusta ponderile și parametri modelului.

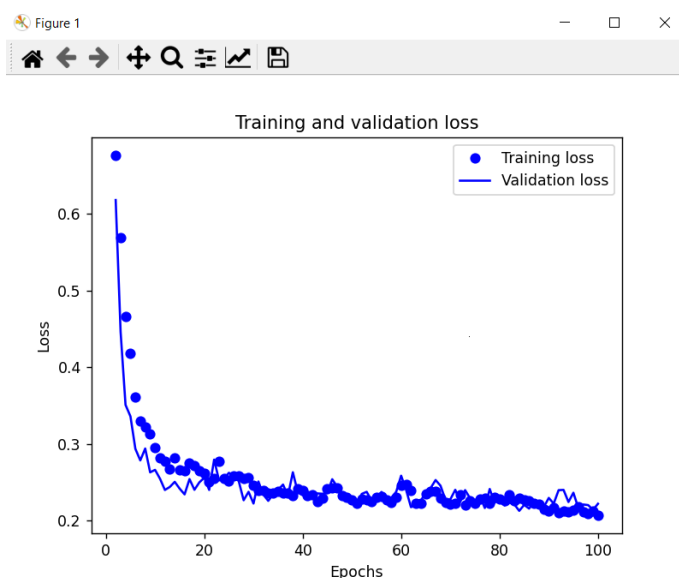


Fig. 12. Funcția de pierdere pe antrenare și validare

Pierdere pe antrenare măsoară progresul modelului în timpul procesului de antrenare. În mod ideal, funcția de loss ar trebui să scadă odată cu creșterea numărului de epoci, demonstrând faptul că modelul reușește să generalizeze și să facă predicții mai precise pe datele de antrenare. Același raționament se aplică și pentru funcția de pierdere pe datele de validare. În figura 4, putem observa tendința de scădere a ambelor funcții de pierdere, pe măsură ce modelul învață. Totodată se poate observa și suprapunerea celor două, demonstrând faptul că modelul are rezultate bune pe date noi și că nu are tendință de overfitting.

O altă metrică folosită pentru evaluarea performanței unui model de clasificare este **acuratețea**, în valoare de 91.35%, reprezentând raportul dintre numărul de exemple clasificate corect și numărul total de exemple.

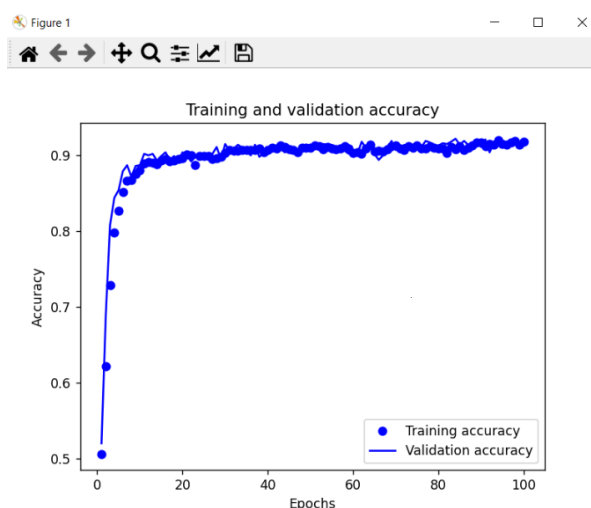


Fig. 13. Acuratețea pentru antrenare și validare

Analizând performanțele modelului rezultat și din perspectiva altor metrici relevante, putem observa faptul că precizia obținută este de 91.53%.

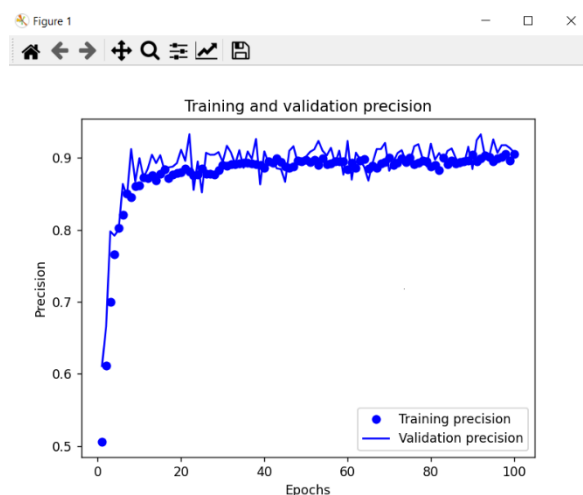


Fig. 14. Precizia pentru antrenare și validare

Precizia reprezintă măsura în care exemplele identificate de model ca phishing sunt corect clasificate, calculată ca raportul dintre numărul exemplor clasificate corect ca phishing și numărul total al exemplor clasificate ca phishing. O precizie ridicată indică faptul că modelul generează puține rezultate fals pozitive, aspect esențial în aplicații critice unde alarmele false trebuie minimizate.

Recall-ul, cu valoarea de 91.31%, reflectă capacitatea modelului de a detecta corect exemplele pozitive reale din totalitatea cazurilor reale de phishing. Această metrică este

crucială pentru evaluarea eficacității unui model în identificarea amenințărilor reale, un recall ridicat indicând faptul că modelul ratează puține cazuri reale de phishing, minimizând astfel erorile fals negative. Cu alte cuvinte, recall-ul mare demonstrează faptul că modelul identifică eficient și consecvent URL-urile periculoase, aspect vital în scenarii de securitate cibernetică unde consecințele unui fals negativ pot fi semnificative.

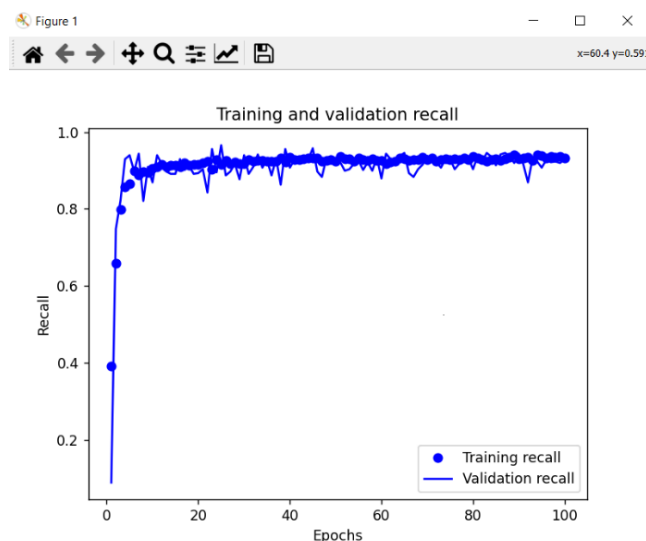


Fig. 15. Recall pentru antrenare și validare

Analizând performanța modelului și din perspectiva metricii **F1-score**, observăm o evoluție pozitivă a acesteia pe parcursul procesului de antrenare. Metrica F1-score este o măsură statistică ce combină armonios precizia și recall-ul, oferind astfel o evaluare echilibrată între capacitatea modelului de a minimiza atât rezultatele fals pozitive, cât și cele fals negative.

Graficul prezentat ilustrează evoluția valorilor F1-score atât pentru setul de antrenare cât și pentru setul de validare, pe o durată de 100 de epoci. Se poate observa că încă din primele 20 de epoci valoarea F1-score crește rapid, ceea ce indică faptul că modelul învață eficient să distingă între URL-urile legitime și cele de tip phishing. Ulterior, metrica F1-score se stabilizează și converge către o valoare ce depășește pragul de 0.9 (91.33%), indicând faptul că modelul reușește să obțină un echilibru foarte bun între precizie și recall, clasificând cu acuratețe majoritatea exemplilor.

Această evoluție sugerează că modelul nu doar că atinge performanțe ridicate, ci și că menține o performanță stabilă, lipsită de variații semnificative după un anumit număr de epoci, indicând o bună generalizare asupra datelor. Astfel, metrica F1-score subliniază eficacitatea și robustețea modelului propus în detectarea amenințărilor de tip phishing, consolidând concluzia că acest model este adecvat pentru utilizarea practică în contexte reale de securitate cibernetică.

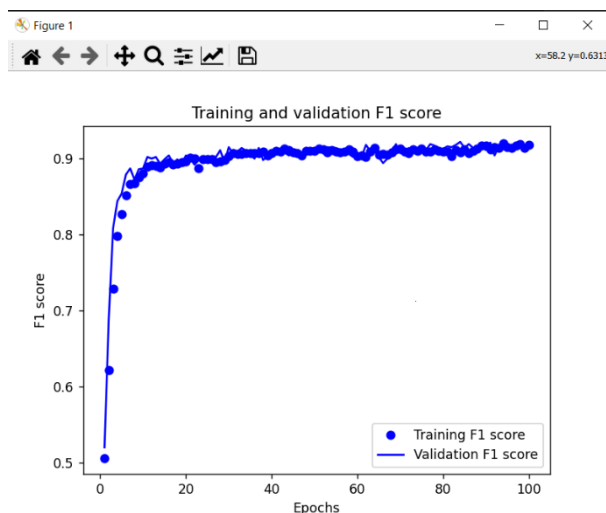


Fig. 16. F1-score pentru antrenare și validare

Toate aceste metrice se pot calcula folosind și **matricea de confuzie**, ce arată performanțele modelului pe setul de date. Elementele acestei matrice sunt de fapt numărul de exemple pentru fiecare categorie. Axa Ox reprezintă eticheta estimată de model, în timp ce axa Oy este eticheta adevărată, astfel încât, pe diagonala principală vom avea clasificările corecte, iar elementele ce se află în afara acesteia vor fi clasificările eronate.

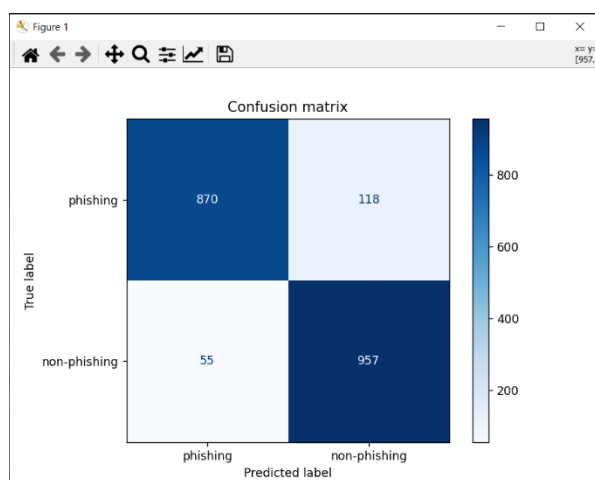


Fig. 17. Matricea de confuzie a modelului

AUC este o metrică extrem de importantă în evaluarea modelelor de clasificare, în special atunci când avem de-a face cu date dezechilibrate sau când vrem să analizăm calitatea modelului dincolo de simpla acuratețe. AUC reprezintă **aria de sub curba ROC (Receiver Operating Characteristic)** și măsoară capacitatea modelului de a distinge între clase. Un scor de **AUC** de 0.5 indică un model care nu poate distinge între clase mai bine decât o

alegere aleatorie, în timp ce un scor de 1.0 indică o separare perfectă între clasele pozitive și negative.

Pentru modelul prezentat, **AUC are valoarea de 96,28%**, ceea ce indică faptul că modelul distinge foarte bine între exemplele de phishing și cele legitime (sau între clasele din problema ta). Un AUC atât de ridicat arată că modelul reușește să mențină o rată scăzută de erori de clasificare indiferent de pragul de decizie ales, fiind robust atât la detecția cazurilor pozitive cât și la respingerea cazurilor negative.

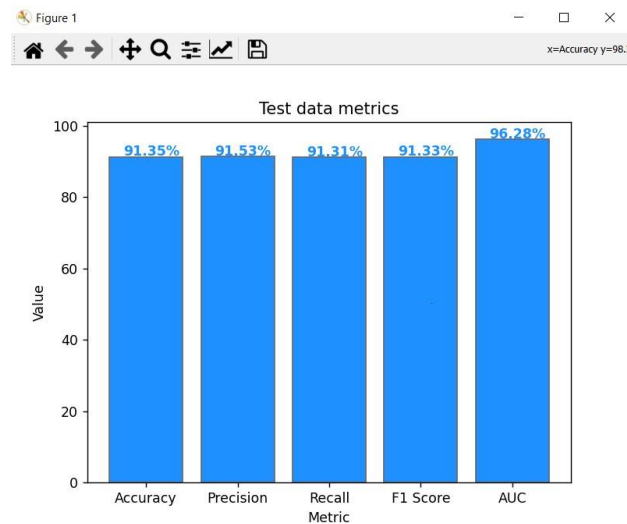


Fig. 18. Matricea de confuzie a modelului

Valorile ridicate și apropiate ale acestor metrici sugerează că modelul nu doar că este precis, dar și echilibrat – nu sacrifică recall-ul pentru a obține o precizie mai mare și invers. Acest aspect este important mai ales în domenii de securitate, unde atât alarmele false (false positive), cât și cazurile ratate (false negative) pot avea consecințe nedorite.

5. Integrarea modelului de învățare automată

5.1 Conectarea extensiei la modelul de învățare automată

Extensiile Chrome sunt aplicații software mici care extind și personalizează funcționalitățile browserului Google Chrome. Aceste extensii oferă utilizatorilor o modalitate de a adăuga caracteristici noi, de a îmbunătăți experiența de navigare sau de a interacționa cu aplicații și servicii externe direct din browser. Ele sunt construite folosind tehnologii web standard, cum ar fi HTML, CSS, și JavaScript, și sunt integrate direct în browser. Extensiile sunt instalate din Chrome Web Store sau local (în timpul dezvoltării). Fiecare extensie are un scop bine definit, de la blocarea reclamelor și gestionarea parolelor, până la analiza site-urilor web sau oferirea de teme personalizate.

Unul dintre componentele importante ale acestei extensii este constituită din fișierul *manifest.json*. Acesta definește informațiile despre aplicație (nume, versiune, descriere) și specifică permisiunile, scripturile și resursele. De asemenea, conține arhitectura modelului și informații despre cum să fie încărcat. Fișierul modelului este construit din detalii despre straturile neuronale, tipurile de activare și alte caracteristici specifice.

Totodată, se pot regăsi și referințe către “*shard-uri*”: *group1-shard1of2.bin* și *group1-shard2of2.bin*, reprezentând date binare cu ponderile pre-antrenate ale modelului. Modalitatea de generare a acestor elemente de machine learning pentru clasificarea binară a fost posibilă prin intermediul utilitarului **tensorflowjs_converter** din cadrul bibliotecii **tensorflowjs** [17] ce permite transformarea modelelor pre-antrenate în formate ce pot fi utilizate direct în aplicație.

Acest lucru este posibil pentru că **tensorflowjs_converter** preia un model antrenat inițial cu TensorFlow (de obicei un model salvat în formatul standard .h5 sau SavedModel) și îl convertește într-un set de fișiere care pot fi încărcate în browser. Printre aceste fișiere se numără *model.json*, care descrie structura și arhitectura modelului neural, și fișierele binare de tip *shard* (cum ar fi *group1-shard1of2.bin*, *group1-shard2of2.bin*), care conțin efectiv valorile ponderilor (weights) învățate în timpul antrenării.

Fișierul *manifest.json* joacă un rol important pentru configurarea extensiei cât și pentru partea de inteligență artificială. În *model.json* sunt detaliate informații precum numărul de straturi, tipul fiecărui strat (de exemplu, dense sau convolutional), funcțiile de activare folosite (relu, sigmoid etc.), precum și conexiunile dintre aceste straturi. Modelul, odată convertit și importat în extensie, permite rularea inferenței direct în browser. Practic, atunci când utilizatorul face o acțiune pentru care extensia a fost concepută (cum ar fi clasificarea unui text sau a unei imagini), browserul va încărca modelul cu ajutorul bibliotecii TensorFlow.js, va procesa inputul utilizatorului prin acest model și va returna rezultatul fără să trimită datele în afara computerului.

Folosirea shard-urilor binare pentru pondere este o soluție tehnică pentru a fragmenta fișierele de dimensiuni mari, astfel încât browserul să le poată încărca mai eficient și să nu depășească limitele impuse de sistemul de fișiere al browserului. Astfel, toate datele necesare pentru model sunt incluse în pachetul extensiei și nu mai depind de servere externe.

Un alt avantaj al acestui tip de integrare este legat de confidențialitate, deoarece datele utilizatorului nu părăsesc dispozitivul local. De asemenea, răspunsul este mult mai rapid, fiindcă nu există latență de rețea. Tot procesul de clasificare sau predicție se întâmplă local, în browser, folosind JavaScript și resursele hardware ale computerului utilizatorului.

TensorFlow.js este o bibliotecă open-source care permite dezvoltarea și rularea modelelor de învățare automată direct în browserul web sau în Node.js, folosind JavaScript. Este o implementare a cunoscutei biblioteci TensorFlow, dar adaptată pentru mediul web. TensorFlow.js permite să construiești, antrenezi și să utilizezi modele ML fără a fi necesar să părăsești ecosistemul JavaScript, făcându-l un instrument foarte puternic pentru aplicațiile web moderne. Printre funcționalitățile de bază ale acestei biblioteci, putem regăsi:

1. **Antrenarea modelelor ML în browser** - TensorFlow.js permite antrenarea modelelor ML direct în browser, folosind datele disponibile pe dispozitivul utilizatorului. Aceasta poate fi o opțiune interesantă pentru aplicațiile care doresc să minimizeze latența și să evite trimiterea datelor către servere externe.
2. **Importul și utilizarea modelelor pre-antrenate** - permite încărcarea și folosirea modelelor pre-antrenate într-o aplicație web. TensorFlow.js este compatibil cu modele antrenate în TensorFlow sau Keras, care pot fi convertite în formatul compatibil cu JavaScript.
3. **Acces la GPU prin WebGL** - TensorFlow.js utilizează WebGL pentru a accelera calculele matematice, permițând procesarea rapidă a datelor chiar și în browser, folosind GPU-ul dispozitivului utilizatorului.

Unul din avantajele folosirii acestei metode este faptul că modelul este rulat direct pe dispozitivul utilizatorului, ceea ce reduce latența și riscurile de securitate legate de transmiterea datelor, dar și viteza de răspuns la clasificare.

Pentru a realiza interfața grafică ce permite vizualizarea procentului de phishing al website-ului accesat de utilizator, s-a folosit framework-ul React.js [18]. React se diferențiază de celelalte framework-uri de frontend prin abordarea sa bazată pe componente și pentru modul său eficient de a actualiza obiectele interfeței. Acesta încorporează stratul de vizualizare în modelul MVC oferind modularitate, facilitând crearea de aplicații web mai rapide. Un motiv pentru care s-a ales folosirea acestei tehnologii pentru partea de frontend a aplicației este datorat eficienței și performanței sale ridicate [19]. Având în vedere faptul că aceasta implică analiza și procesarea url-urilor, precum și clasificarea binară prin învățarea automată, este important ca rezultatul să fie o interfață rapidă și reactivă.

5.2 Testarea funcționalității integrate

Testarea este o etapă critică în dezvoltarea extensiei, având rolul de a asigura că modelul de învățare automată funcționează corect în cadrul extensiei și că detectează în mod precis atacurile de tip phishing. Procesul implică verificarea componentelor individuale, precum și a interacțiunii dintre extensie și model.

Câteva tipuri de teste pentru extensia integrată ar putea fi:

- I. Teste funcționale - se asigură că fiecare funcționalitate a extensiei funcționează conform așteptărilor, cum ar fi:
 - detectarea corectă a URL-urilor de phishing
 - afișarea alertelor către utilizator
 - colectarea și trimiterea feedback-ului utilizatorului
- II. Teste de integrare - se verifică interacțiunea dintre extensie și modelul de învățare automată și se testează comunicarea între frontend (extensia din browser) și backend (modelul ML local sau pe server)
- III. Teste de performanță - evaluează viteza cu care extensia analizează URL-urile și oferă rezultatele și analizează consumul de resurse al extensiei
- IV. Teste de utilizare (UX/UI) - se verifică dacă extensia este ușor de utilizat și dacă interfața este clară pentru utilizatori

Pentru a evalua performanța modelului de detecție phishing, s-a măsurat timpul necesar pentru clasificarea fiecărui URL, folosind un set de 10 adrese (atât legitime cât și phishing), fiecare fiind evaluată de 5 ori pentru robustețe statistică. **Timpul mediu de inferență** obținut pentru o clasificare a fost de **64 milisecunde**. Aceasta înseamnă că modelul poate decide aproape instantaneu dacă un URL este phishing sau legitim, făcându-l potrivit pentru aplicații în timp real, cum ar fi extensii de browser.

Graficul din figura de mai jos redă vizual performanța modelului din perspectiva timpului de inferență pentru fiecare URL testat, oferind o imagine de ansamblu atât asupra vitezei de clasificare, cât și asupra stabilității acestui proces. Pe axa orizontală sunt reprezentate individual cele zece URL-uri evaluate, iar pe axa verticală este indicat timpul mediu necesar clasificării fiecărui URL, exprimat în milisecunde. Fiecare bară corespunde timpului mediu de inferență calculat pe baza a cinci rulări succesive pentru același URL, ceea ce permite atenuarea efectului unor fluctuații punctuale ale resurselor de sistem sau ale contextului de execuție.

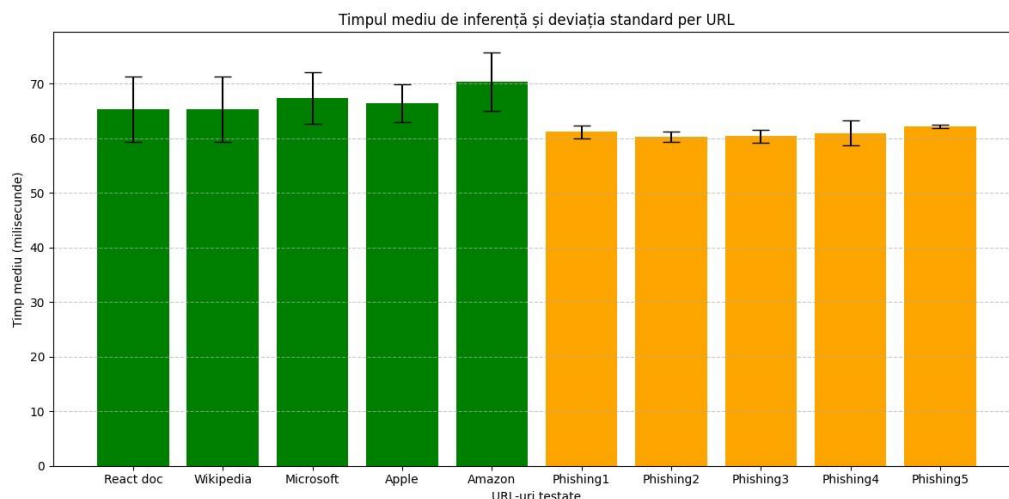


Fig. 19. Timpul mediu de inferență și deviația standard pe un eșantion de 10 URL-uri

Elementul distinctiv al graficului este reprezentat de barele de eroare suprapuse peste fiecare coloană, acestea ilustrând deviația standard asociată setului de măsurători pentru fiecare URL în parte. Astfel, barele de eroare reflectă variabilitatea internă a timpilor de inferență și permit o evaluare rapidă a consistenței performanței modelului de la o rulare la alta. Observăm că, pentru majoritatea URL-urilor, lungimea segmentelor de eroare este redusă, ceea ce indică o dispersie mică a valorilor și, implicit, un comportament stabil al modelului. În situațiile în care segmentele de eroare sunt semnificativ mai extinse, acest lucru semnalează o variabilitate mai ridicată a timpului de execuție, sugerând că în acele cazuri clasificarea poate fi influențată de factori suplimentari precum conținutul particular al URL-ului sau starea sistemului în momentul execuției.

În contextul de față, graficul subliniază nu doar faptul că modelul este rapid (valoarea medie fiind 64 ms), ci și că acest timp de răspuns este consecvent pentru majoritatea cazurilor testate, consolidând astfel concluzia că soluția este adecvată pentru detecția phishing în timp real.

Modalitatea de accesare a extensiei se face prin widget-ul de extensii al browser-ului Chrome, urmând ca o interfață în miniatură să fie afișată în partea din dreapta sus a ecranului cu toate informațiile necesare furnizate, așa cum se poate observa în figura de mai jos.

Extensia creată, WebSentinel [20], are rolul esențial de a afișa, în primul rând, predicția modelului de clasificare binară, precum și nivelul de încredere aferent fiecărei predicții, exprimat procentual. În contextul modelelor de învățare automată care rezolvă probleme de clasificare binară, ieșirea modelului nu este, în general, direct o etichetă ("phishing" sau "legitim"), ci o probabilitate asociată fiecărei clase. Această probabilitate reprezintă gradul de certitudine al modelului față de apartenența exemplului analizat la una dintre cele două clase posibile.

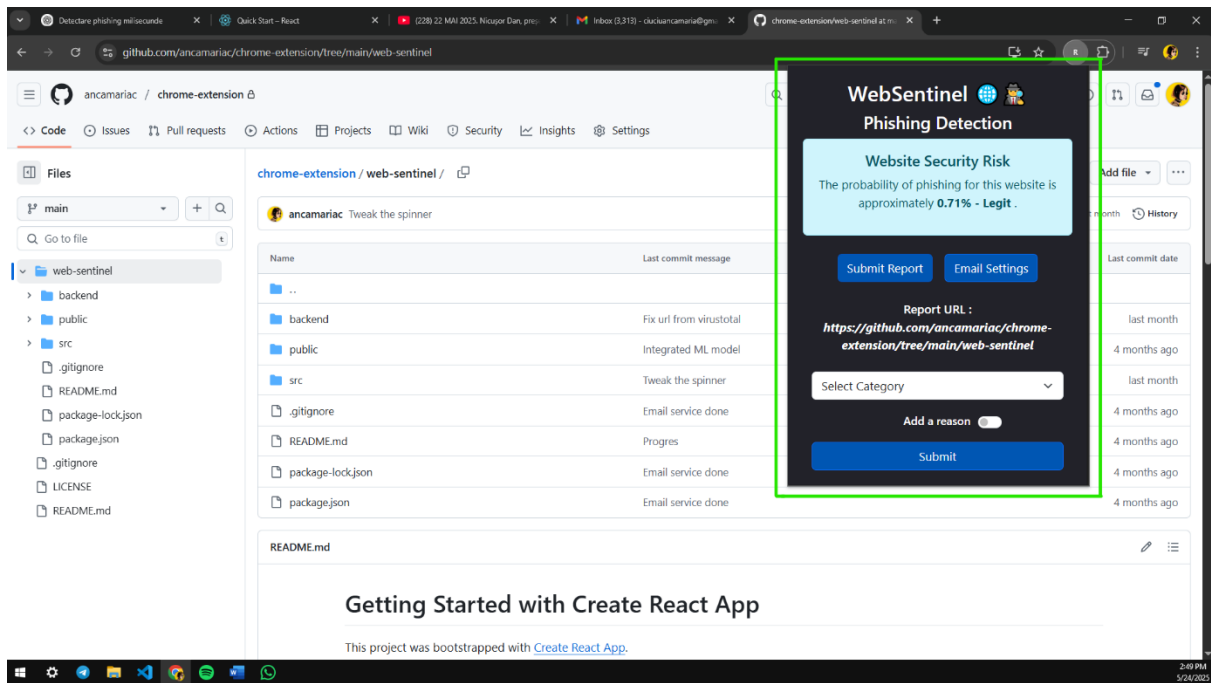


Fig. 20. Interfața extensiei web

Probabilitatea menționată este, de regulă, rezultatul aplicării unei funcții de activare sigmoid asupra scorului brut (logit) calculat de model. Funcția sigmoid este folosită pe scară largă în logistic regression, cât și în rețele neuronale, deoarece are proprietatea de a transforma orice valoare reală (pozitivă sau negativă) într-un rezultat cuprins între 0 și 1. Acest rezultat poate fi astfel interpretat ca o probabilitate.

În termeni matematici, dacă notăm cu z scorul sau logitul produs de model pe baza caracteristicilor de intrare X , atunci probabilitatea ca exemplul analizat să aparțină clasei pozitive ($y = 1$) este calculată folosind funcția sigmoid, astfel cum este ilustrat în formula (1):

$$P(y = 1|X) = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

Astfel, pe baza valorii de probabilitate returnate de model, aplicația efectuează o clasificare automată a URL-ului analizat. Dacă probabilitatea este mai mare sau egală cu 0.5, adică există o certitudine ridicată că exemplul aparține clasei de phishing, sistemul va atribui eticheta „**Phishing**”. În schimb, dacă probabilitatea este egală sau mai mică de 0.3, adică modelul are un grad ridicat de încredere că URL-ul este legitim, se va afișa eticheta „**Legit**”. Pentru valorile intermediare, unde probabilitatea este cuprinsă între 0.3 și 0.5, sistemul semnalează o zonă de incertitudine, marcând clasificarea cu mesajul „**Caution Advised**” (atenție recomandată).

Decizia de a utiliza aceste praguri a fost luată în urma unei analize asupra distribuției scorurilor de probabilitate generate de model pe un set relevant de date de test. Prin studierea acestor rezultate, s-a observat că valorile de probabilitate aflate sub 0.3 sau peste 0.5 corespund cu un grad ridicat de certitudine al predicției, în timp ce intervalul intermediar reflectă situații mai ambigue, unde modelul nu poate oferi o decizie fermă. Această abordare permite atât reducerea riscului de alarme false, cât și o informare suplimentară a utilizatorului în cazurile în care incertitudinea este crescută, contribuind astfel la transparența și utilitatea practică a sistemului de detecție.

5.3 Prezentarea aplicației și a funcționalităților

Extensia WebSentinel este o soluție modernă de detecție a phishing-ului, proiectată pentru a oferi utilizatorului protecție în timp real direct în browser. Interfața extensiei este intuitivă și informativă: la accesarea unui site, WebSentinel analizează automat adresa curentă și afișează un sumar al riscului de securitate într-un mod clar, vizibil în partea superioară a interfeței.

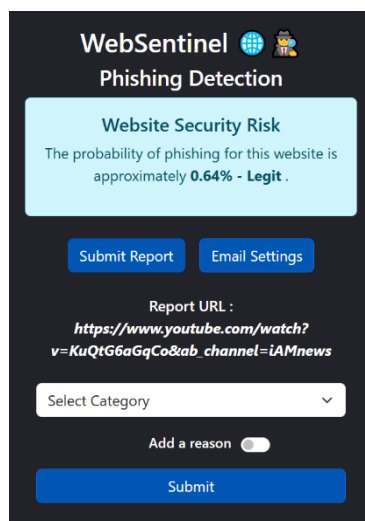


Fig. 21. Interfața extensiei WebSentinel - Submit Report

Unul dintre principalele avantaje ale acestei extensii este faptul că nu se limitează la o clasificare binară „phishing”/„legitim”, ci oferă și nivelul de probabilitate asociat fiecărei predicții, exprimat procentual. Acest aspect permite utilizatorului să înțeleagă cât de sigur este modelul pe baza datelor analizate. De exemplu, în captura de ecran prezentată, extensia indică o probabilitate de phishing de doar 0,64 %, clasificând astfel website-ul drept „Legit”.

Pe lângă detecția automată, extensia pune la dispoziția utilizatorului funcționalități suplimentare de raportare și personalizare. Butonul „**Submit Report**” permite raportarea rapidă a site-urilor suspecte către administrator sau către o bază de date centralizată, în timp ce „**Email Settings**” oferă posibilitatea configurării notificărilor sau alertelor pe email, personalizând astfel nivelul de protecție.

În partea inferioară a interfeței, utilizatorul are opțiunea de a selecta categoria pentru raportul său, precum și de a adăuga un motiv specific folosind switch-ul „**Add a reason**”. Această abordare asigură colectarea de feedback detaliat și relevant despre posibilele amenințări întâlnite în navigare. Această este **un câmp pentru justificare** astfel că utilizatorii pot oferi motive pentru care consideră site-ul raportat ca fiind periculos/sigur precum și **categorizarea riscului** reprezentată printr-un dropdown conținând opțiunile: Legitimate/Phishing. Odată creat acest raport, se va trimite un mail către o adresă creată cu scopul de a face o verificare manuală a website-ului de către echipa administrativă, această aplicație fiind destinată companiilor care vor să asigure un mediu sigur angajaților în ceea ce privește navigarea web.

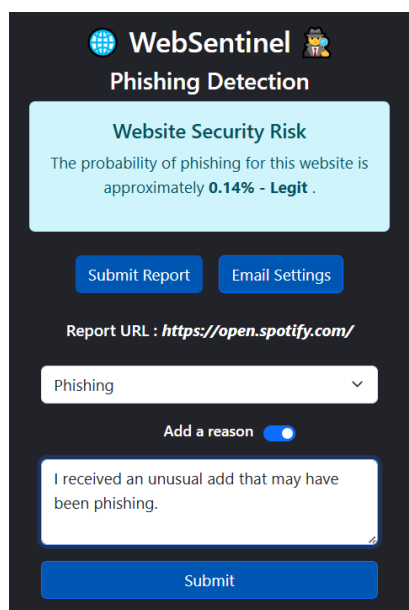


Fig. 22. Interfața extensiei WebSentinel - Add a reason

Odată ce un utilizator raportează un site, extensia de browser va salva această acțiune local, în **localStorage** pentru a preveni raportarea repetată a aceluiași site. Această funcționalitate este esențială pentru a evita redundanța și a asigura că fiecare site raportat este verificat doar o singură dată de utilizator.

Un alt element important al extensiei WebSentinel este implementarea unui ecran de încărcare (loading screen) ce apare în momentul în care utilizatorul trimite un raport („Submit Report”). Această funcționalitate asigură un feedback vizual imediat, informând utilizatorul că acțiunea sa este procesată și evitând astfel impresia de „blocare” sau lipsă de reacție din partea aplicației. Integrarea unui astfel de mecanism de feedback contribuie la o interfață mai prietenoasă și la o experiență fluentă, specifică aplicațiilor responsive și moderne. Prin acest detaliu, extensia transmite clar că fiecare acțiune a utilizatorului este preluată și gestionată prompt, ceea ce sporește încrederea în instrument și în eficiența fluxului de raportare a incidentelor.

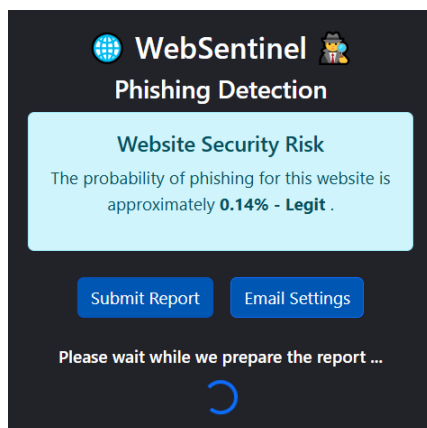


Fig. 23. Interfața extensiei WebSentinel - Raport în așteptare

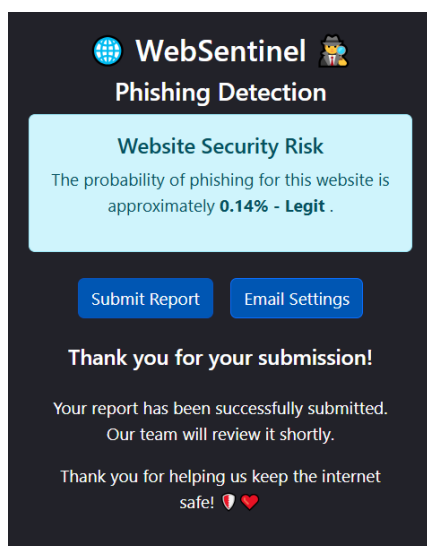


Fig. 23. Interfața extensiei WebSentinel - Raport trimis

Extensia WebSentinel integrează, de asemenea, o secțiune dedicată setărilor de email, unde utilizatorii pot specifica adresa către care vor fi trimise rapoartele de phishing generate din cadrul extensiei. Această funcționalitate este extrem de utilă atât în mediul personal, cât și în contexte organizaționale, permițând direcționarea automată a incidentelor raportate către departamentele sau persoanele responsabile cu securitatea IT.

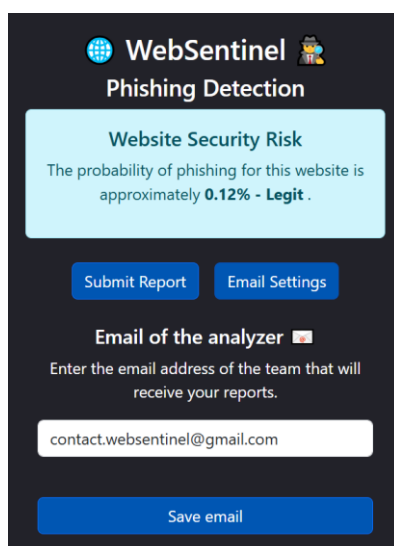
Toate aceste elemente, combinate cu un design vizual intuitiv, fac din WebSentinel o extensie completă, potrivită atât pentru utilizatorii obișnuiți, cât și pentru mediile profesionale în care securitatea navigării este esențială.

5.4 Raportarea URL-urilor

Procesul de raportare a URL-urilor presupune interacțiunea utilizatorului cu extensia pentru a semnală un site pe care îl consideră periculos sau care poate reprezenta o amenințare, cum ar fi un site de phishing, scam sau malware. De asemenea, este posibil ca unele website-uri să fie clasificate ca fiind phishing deși nu prezintă niciun risc. După raportarea site-ului, extensia poate trimite informațiile către un server, care poate analiza URL-ul și trimite un e-mail echipei de securitate sau administratorului pentru a lua măsuri suplimentare.

Un **studiu de caz** în acest sens ar putea viza situația în care un angajat, în timpul navigării online, accesează un site care pare a fi un phishing sau o tentativă de fraudă. În loc să ignore riscurile, angajatul poate utiliza extensia de securitate pentru a raporta site-ul. Aceasta trimite imediat informațiile către echipa IT, iar un tehnician din departamentul IT poate analiza rapid statusul URL-ului raportat și poate decide dacă trebuie să fie blocat pe rețea sau dacă mai multe măsuri de securitate sunt necesare. În acest fel, organizația se asigură că sunt identificate rapid site-urile periculoase, iar angajații sunt protejați de posibilele atacuri cibernetice.

Prin intermediul aplicației dezvoltate, WebSentinel, utilizatorii pot raporta URL-urile, acest proces concretizându-se prin trimiterea automată a unui mail folosindu-se serverul de mailuri Mailjet pentru a asigura un flux eficient și securizat. Utilizatorii pot raporta URL-uri direct prin extensia creată, iar datele raportate, inclusiv URL-ul și informațiile adiționale, sunt transmise către serverul backend. Acesta procesează raportul și folosește Mailjet pentru a trimite notificări către echipa de securitate. Integrarea cu Mailjet a fost realizată prin configurarea cheilor API și utilizarea SDK-ului oficial, generând e-mailuri detaliate care includ informații precum URL-ul raportat, utilizatorul care l-a raportat și momentul raportării. Acest sistem asigură o comunicare rapidă și permite gestionarea eficientă a incidentelor de securitate cibernetică. Astfel, după cum poate fi observat și în imaginea de mai jos, utilizatorul poate seta adresa de email ce va primi raportul pentru URL-ul în cauză.



The image shows a mobile application interface for 'WebSentinel Phishing Detection'. At the top, there's a header with a globe icon, the text 'WebSentinel', and a small robot icon. Below this is the title 'Phishing Detection'. A light blue box contains the text 'Website Security Risk' and 'The probability of phishing for this website is approximately 0.12% - Legit'. Below this box are two buttons: 'Submit Report' and 'Email Settings'. Underneath these buttons is the section 'Email of the analyzer' with a small flag icon. It says 'Enter the email address of the team that will receive your reports.' Below this is a text input field containing 'contact.websentinel@gmail.com'. At the bottom is a blue button labeled 'Save email'.

Fig. 24. Interfața extensiei WebSentinel - Email settings

Reiterând scenariul descris anterior cu angajații unei companii, extensia poate fi personalizată cu o adresă de email cu scopul de a notifica departamentul de IT responsabil. Mailul trimis conține informațiile din raport precum și o analiză realizată de VirusTotal.

VirusTotal [21] este o platformă online de securitate cibernetică care permite utilizatorilor să analizeze fișiere și URL-uri pentru a detecta potențiale amenințări malware și alte tipuri de activități malițioase. Creată inițial în 2004 și ulterior achiziționată de Google în 2012, VirusTotal integrează mai multe motoare antivirus, instrumente de analiză a comportamentului și baze de date privind amenințările cibernetice.

În ceea ce privește analiza site-urilor, VirusTotal efectuează scanări pentru a identifica dacă un URL este asociat cu phishing, malware sau alte activități malițioase. Rapoartele generate includ informații precum starea de siguranță a site-ului, scoruri atribuite de diverse motoare de securitate, reputația domeniului și detalii tehnice, cum ar fi locația serverului și datele despre certificatul SSL/TLS. De asemenea, platforma detectează comportamente suspecte, cum ar fi redirectări neautorizate, încărcarea de scripturi periculoase sau comunicarea cu servere compromise.

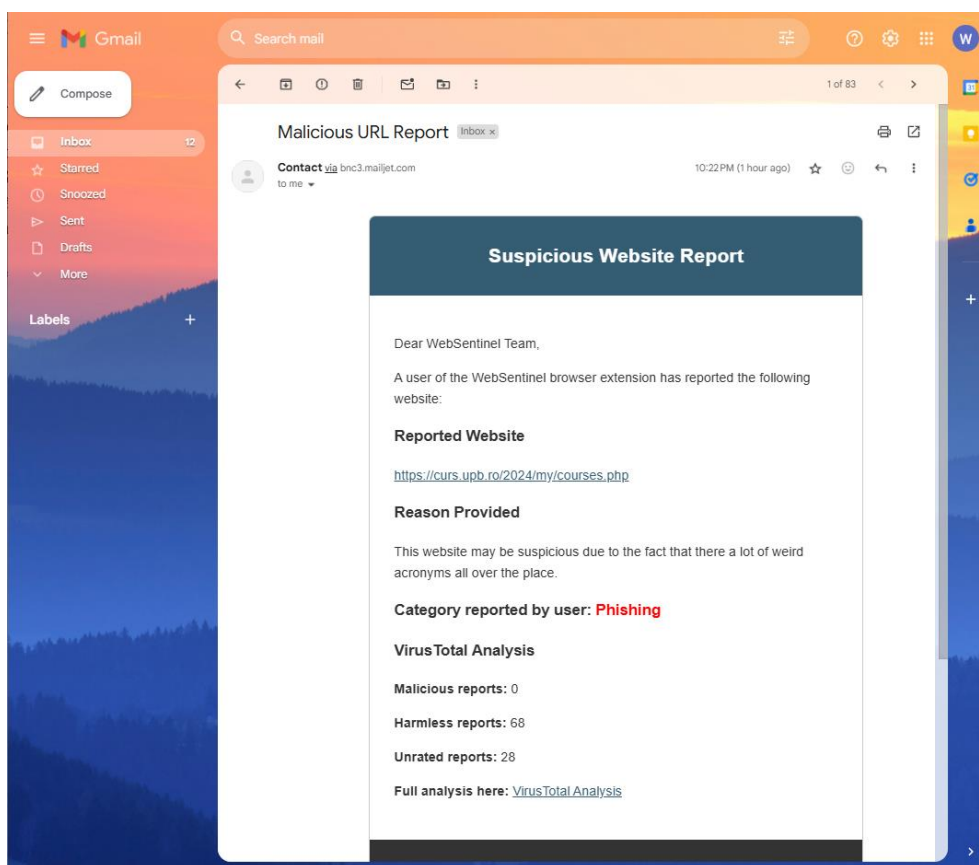


Fig. 25 . Mailul primit în urma raportării

Conținutul raportului include secțiuni distincte care ajută la o înțelegere rapidă și eficientă a incidentului semnalat:

- **Reported Website:** Această secțiune afișează URL-ul exact al site-ului raportat, facilitând accesul direct și rapid la pagina respectivă.
- **Reason Provided:** În exemplul prezentat, motivul care a fost completat în extensie indică faptul că pagina conține acronime ciudate sau neobișnuite, ceea ce poate fi interpretat drept un indicator al unei tentative de phishing.
- **Category reported by user:** Această secțiune prezintă categoria aleasă de utilizator în momentul raportării. În cazul de față, categoria selectată este **Phishing**, marcată în mod distinct prin culoarea roșie pentru a sublinia gravitatea și tipul amenințării.
- **VirusTotal Analysis:** O componentă extrem de utilă a raportului este secțiunea dedicată analizei automate oferite de VirusTotal, care oferă rapid o perspectivă adițională și obiectivă asupra legitimității site-ului raportat. Rezultatele analizei includ:
 - Numărul rapoartelor **Malicious** (în acest caz 0),
 - Numărul rapoartelor **Harmless** (în acest caz 68),
 - Numărul rapoartelor **Unrated** (în acest caz 28),
 - și un link direct către **analiza completă** pe platforma VirusTotal.

Această abordare combinată a feedback-ului oferit de utilizatori și a analizei automate VirusTotal creează un raport comprehensiv, util echipei de securitate pentru evaluarea rapidă a amenințărilor și luarea deciziilor informate cu privire la acțiunile ulterioare necesare. Astfel, emailul generat de WebSentinel este conceput pentru a maximiza eficiența procesului de gestionare a incidentelor de securitate.

6. Concluzii și dezvoltări ulterioare

Integrarea feedback-ului utilizatorilor constituie un element cheie în procesul de îmbunătățire continuă a performanței sistemelor de detecție a phishing-ului. În mod particular, aplicațiile de securitate care interacționează direct cu utilizatorii beneficiază de pe urma acestei implicări, deoarece utilizatorii finali au posibilitatea de a semnala atât alarmele false, cât și tentativele reale de phishing care au trecut neobservate de către modelul automatizat. Colectarea acestui feedback, într-o manieră anonimă și respectând principiile etice, reprezintă o sursă valoroasă de informații care permite recalibrarea și adaptarea algoritmilor de detecție la dinamica tot mai accentuată a amenințărilor cibernetice.

O abordare eficientă presupune existența unui mecanism integrat în aplicație prin care utilizatorii să poată raporta rapid orice eroare de clasificare. Astfel, aceștia pot marca un site sau un mesaj considerat legitim ca fiind, de fapt, suspect, sau pot semnala situațiile în care sistemul emite un avertisment nejustificat. Aceste raportări sunt apoi colectate și centralizate într-o bază de date, urmând să fie analizate și valorificate în procesele ulterioare de reantrenare ale modelului. Este deosebit de important ca sistemul să poată diferenția între feedback-ul provenit de la utilizatori obișnuiți și cel furnizat de persoane cu un nivel ridicat de expertiză, cum ar fi administratorii de sistem sau specialiștii IT, pentru a reduce impactul eventualelor raportări eronate sau abuzive.

Implementarea unui astfel de mecanism aduce mai multe avantaje semnificative. În primul rând, implică utilizatorii în procesul de securizare, transformându-i din simpli beneficiari în participanți activi la îmbunătățirea sistemului. În al doilea rând, feedback-ul colectat oferă o perspectivă directă asupra situațiilor concrete în care algoritmul nu funcționează optim, facilitând identificarea și corectarea rapidă a acestor vulnerabilități. Prin includerea exemplurilor raportate în seturile de date folosite pentru reantrenarea modelului, acesta devine mai bine adaptat la noile tactici și strategii folosite de atacatori, crescându-și astfel gradul de acuratețe și robustețe.

Totodată, gestionarea feedback-ului trebuie să țină cont de aspecte legate de confidențialitatea și protecția datelor personale ale utilizatorilor. Este esențial ca procesul de colectare să se realizeze într-un mod anonim, astfel încât să nu permită identificarea indivizilor sau monitorizarea comportamentului acestora, respectând astfel cerințele legale și etice privind protecția datelor.

În concluzie, integrarea feedback-ului utilizatorilor în sistemele de detecție a phishing-ului oferă nu doar o sursă suplimentară de date pentru îmbunătățirea performanțelor algoritmilor, ci și o modalitate de a transforma sistemul într-un instrument adaptiv, capabil să răspundă prompt la evoluția amenințărilor din mediul cibernetic și să ofere o protecție sporită pentru toți utilizatorii.

Incremental learning este o abordare în domeniul învățării automate, prin care un model de inteligență artificială dobândește date noi progresiv, păstrând și îmbunătățind baza sa de cunoștințe existente.

Această abordare permite modelelor să încorporeze treptat noi date, fără a pierde informațiile dobândite anterior, abordând provocările legate de **catastrophic forgetting** [22] (uitarea catastrofală). Pentru a implementa învățarea incrementală eficientă, rețelele neuronale trebuie să fie proiectate modular, astfel încât să poată adăuga și actualiza componente specifice fără a afecta părțile deja antrenate.

Totodată, o altă tehnică întâlnită este cea a straturilor congelate (**frozen layers** [23]). Straturile inițiale (cele mai apropiate de intrare) sunt deseori "congelate" pentru a păstra caracteristicile fundamentale extrase din datele anterioare. Straturile superioare sunt ajustate pentru a integra noile informații. Ea implică păstrarea constantă a parametrilor (ponderilor și biasurilor) unor straturi ale rețelei neuronale în timpul procesului de re-antrenare. Acest lucru este util mai ales atunci când un model pre-antrenat este reutilizat pe un nou set de date sau pentru a preveni pierderea cunoștințelor dobândite anterior.

Într-o rețea neuronală, straturile sunt ca niște blocuri de construcție în care are loc procesul de învățare. Fiecare strat este compus din neuroni care captează diferite caracteristici din datele de intrare. De exemplu, într-o rețea neuronală convoluțională (CNN) utilizată pentru recunoașterea imaginilor, straturile inițiale detectează caracteristici simple, cum ar fi marginile și texturile, în timp ce straturile mai profunde captează modele mai complexe, precum formele și obiectele.

De obicei, în scenariile de învățare continuă, straturile inițiale ale unui model pre-antrenat au învățat să detecteze caracteristici generice utile pentru o gamă largă de sarcini. De exemplu, într-un model antrenat pe un set mare de imagini, primele câteva straturi vor învăța să recunoască margini, colțuri și texturi. Aceste caracteristici sunt universale, astfel că, de cele mai multe ori, nu este necesară antrenarea pentru o sarcină specifică. Înghețarea acestor straturi permite concentrarea pe ajustarea straturilor mai profunde, care sunt mai specializate pentru noua sarcină. Astfel, această tehnică este folosită spre exemplu în analiza imaginilor medicale, monitorizarea agricolă și recunoașterea obiectelor în vehicule autonome. Practic, modalitatea prin care se îngheață un strat ar fi prin manipularea atributului "trainable" ce va fi setat pe fals.

Optimizarea este esențială în procesul de învățare a rețelelor neuronale, mai ales în scenarii de învățare incrementală, unde modelul trebuie să încorporeze noi cunoștințe fără a pierde cele existente. În continuare, dezvoltăm detaliat două tehnici importante: Stochastic Gradient Descent (SGD) cu rate de învățare adaptive și Elastic Weight Consolidation (EWC), ambele având aplicații semnificative în acest context.

I. Stochastic Gradient Descent (SGD) cu rate de învățare adaptive

SGD este unul dintre cei mai utilizați algoritmi de optimizare. Funcționează prin actualizarea ponderilor modelului la fiecare pas de antrenare, utilizând un subset (mini-lot) din datele de antrenare.

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta) \quad (1)$$

unde:

- θ_t este vectorul de ponderi la pasul t
- η este rata de învățare
- $\nabla_{\theta} J(\theta)$ este gradientul funcției de pierdere

Ratele de învățare fixe pot duce fie la supraajustare (overfitting), fie la o învățare lentă. Algoritmii de optimizare cu rate de învățare adaptive, precum **Adam** și **RMSProp**, oferă o soluție eficientă:

- **Adam (Adaptive Moment Estimation):** combină avantajele SGD și ale momentului, calculând rate de învățare adaptative pentru fiecare parametru. Este ideal pentru date sparse și scenarii cu zgomot ridicat.
- **RMSProp (Root Mean Square Propagation):** ajustează rata de învățare pentru fiecare parametru, bazându-se pe media pătratică mobilă a gradientului. Funcționează bine în scenarii cu date dinamice.

II. Elastic Weight Consolidation (EWC)

EWC este o metodă care combate uitarea catastrofică penalizând modificările semnificative ale ponderilor critice pentru sarcinile învățate anterior. Se bazează pe ideea că anumite ponderi sunt mai importante decât altele și ar trebui conservate în timpul antrenării pe noi date.

Funcția de pierdere a EWC este:

$$L(\theta) = L_{\text{nou}}(\theta) + \lambda \sum_i F_i (\theta_i - \theta_i^{\text{vechi}})^2 \quad (2)$$

unde:

- $L_{\text{nou}}(\theta)$: funcția de pierdere pentru sarcina nouă
- λ : hiperparametru care controlează cât de strict penalizează modificările
- F_i : matricea Fisher, reprezentând importanța fiecărui parametru θ_i

Această metodă oferă un echilibru între adaptabilitatea la noi date și conservarea cunoștințelor existente, fiind ideală în aplicații precum recunoașterea imaginilor, analiza datelor medicale sau procesarea limbajului natural. Totuși, succesul EWC depinde de un compromis bine ajustat între stabilitate (păstrarea cunoștințelor vechi) și plasticitate (învățarea noilor cunoștințe), precum și de capacitatea modelului de a gestiona resursele computaționale asociate calculelor adiționale.

În scenariile moderne de detectare a site-urilor de tip phishing, rolul utilizatorului nu se limitează doar la cel de beneficiar pasiv al soluțiilor de securitate, ci devine esențial în dinamica procesului de învățare continuă a sistemului. Implicarea activă a utilizatorilor, care pot oferi constant feedback referitor la acuratețea verdictului emis de model, reprezintă una dintre cele mai eficiente modalități de adaptare rapidă la noile strategii și tehnici utilizate de atacatori. Această participare directă poate lua forma raportării unui verdict eronat – de exemplu, atunci când un site considerat legitim este de fapt o tentativă de phishing sofisticată sau, din contră, când o pagină inofensivă este marcată greșit ca fiind periculoasă. Astfel de intervenții furnizează un flux constant de date relevante, care devin extrem de utile pentru recalibrarea, rafinarea și perfecționarea continuă a algoritmului de detecție.

În practică, implementarea unui mecanism automatizat de colectare, prelucrare și integrare a acestui feedback implică o serie de etape tehnice complexe, menite să asigure atât calitatea datelor colectate, cât și fiabilitatea deciziilor ulterioare ale sistemului. În prima fază, feedback-ul utilizatorilor este supus unui proces riguros de preprocesare, care urmărește să elimine zgomotul de fond generat de raportări neclare sau incomplete, să evite duplicarea informațiilor și să valideze autenticitatea semnalărilor prin corelarea acestora cu surse externe de încredere. Acest proces de verificare poate presupune consultarea unor baze de date recunoscute, cum ar fi listele negre sau albe (blacklist/whitelist) internaționale, precum și rapoartele agregate de servicii specializate, cum ar fi VirusTotal, cu scopul de a crește acuratețea și relevanța datelor introduse în sistem.

Robustețea procesului este sporită prin agregarea feedback-ului la nivel de consens, ceea ce presupune ca o decizie să fie luată doar atunci când un număr suficient de utilizatori semnalează aceeași problemă sau când se constată o convergență a opiniilor. În acest mod, riscul de a introduce în model erori generate de raportări individuale, izolate sau chiar intenționat false este considerabil redus. Totuși, există situații în care opiniile utilizatorilor pot fi divergente sau contradictorii, iar în astfel de cazuri sistemul poate recurge la algoritmi avansați de crowdsourcing, capabili să evalueze gradul de încredere asociat fiecărei etichete de feedback. Acești algoritmi iau în considerare istoricul raportărilor anterioare ale utilizatorilor, acuratețea deciziilor precedente și chiar gradul de expertiză demonstrat de fiecare participant, astfel încât să poată filtra automat semnalele autentice de cele suspecte sau malițioase.

Din perspectivă tehnică, un element fundamental în asigurarea eficienței și continuității procesului de învățare automată este utilizarea unor tehnici specializate precum Elastic Weight Consolidation (EWC). Această metodă este menită să prevină fenomenul cunoscut sub numele de „catastrophic forgetting”, care apare atunci când modelul, în încercarea de a se adapta la date noi, pierde cunoștințele dobândite anterior prin reantrenare. Elastic Weight Consolidation abordează această problemă penalizând modificările semnificative ale parametrilor critici ai rețelei neuronale, menținând astfel echilibrul între învățarea noilor tipare de phishing identificate prin feedback și păstrarea performanțelor obținute pe seturile de date istorice.

Odată ce noile date au fost integrate în sistem și modelul a fost reantrenat, este esențial să se realizeze un proces riguros de revalidare. Această etapă implică testarea modelului atât

pe seturile de date istorice, cât și pe cele noi provenite din feedback-ul recent, pentru a se asigura că ajustările aduse nu compromit performanța globală a sistemului. Validarea pe multiple tipuri de date oferă o garanție suplimentară că evoluția modelului se realizează controlat și că acesta continuă să răspundă eficient la provocările impuse de peisajul dinamic al amenințărilor cibernetice.

Pe termen lung, acest flux de lucru iterativ, bazat pe integrarea constantă a feedbackului utilizatorilor, asigură nu doar îmbunătățirea ratelor de detecție a atacurilor de tip phishing, ci și reducerea semnificativă a alarmelor false, care pot afecta încrederea și experiența finală a utilizatorilor. De asemenea, adaptabilitatea crescută a modelului permite sistemului să țină pasul cu evoluția rapidă a tehnicilor utilizate de atacatori, răspunzând prompt la noile amenințări, fără a fi necesară intervenția manuală constantă a echipelor tehnice.

În concluzie, integrarea sistematică și automatizată a feedbackului utilizatorilor în procesele de actualizare și optimizare a modelelor de detecție nu doar că sporește robustețea și eficiența soluțiilor de securitate, ci creează și un cadru scalabil și adaptiv, capabil să facă față cu succes provocărilor unui mediu cibernetic în continuă schimbare. Astfel, colaborarea inteligentă dintre utilizator și tehnologie devine unul dintre pilonii fundamentali ai protecției eficiente împotriva phishing-ului.

În cadrul acestei lucrări, a fost concepută și evaluată o soluție software inovatoare pentru detecția phishing-ului sub forma unei extensii de browser, WebSentinel, bazată pe rețele neuronale artificiale și principii moderne de învățare automată. Modelul a fost antrenat, optimizat și validat folosind tehnici avansate precum Grid Search pentru selecția hiperparametrilor, împărțirea riguroasă a setului de date în subseturi de antrenare, validare și test, precum și evaluarea performanțelor pe metriche relevante pentru contextul securității informatice.

Unul dintre principalele obiective ale proiectului a fost atingerea unei performanțe cât mai ridicate în detecția site-urilor de tip phishing, fără a compromite viteza de răspuns sau experiența utilizatorului. În urma testării pe un set reprezentativ de adrese web - 10 URL-uri distincte, acoperind atât exemple legitime, cât și site-uri de phishing preluate din surse consacrate - și rulării repetate de cinci ori pentru fiecare exemplu, s-au obținut rezultate robuste din punct de vedere statistic. **Acuratețea finală a modelului s-a situat la valoarea de 91.35%**, reflectând capacitatea sistemului de a oferi predicții corecte într-o proporție covârșitoare a cazurilor întâlnite în practică.

O contribuție majoră adusă de această lucrare constă în analiza detaliată a timpului de inferență al modelului, parametru critic pentru orice sistem de securitate ce operează în timp real. Pentru fiecare rulare, s-a măsurat timpul efectiv necesar pentru procesarea și clasificarea unui URL, rezultând un **timp mediu de inferență de doar 64 milisecunde**. Acest rezultat plasează extensia WebSentinel în categoria soluțiilor eficiente din punct de vedere computațional, capabile să ofere protecție instantanee fără a afecta negativ performanța generală a browserului sau experiența utilizatorului.

Pe lângă aceste aspecte tehnice, extensia a fost dezvoltată cu o atenție deosebită acordată utilizabilității și transparenței. Interfața prietenoasă permite utilizatorului să primească nu doar

verdictul automat al modelului, ci și un scor procentual de încredere, precum și posibilitatea de a raporta rapid orice site suspect către departamentul IT sau către adresa de email configurată. Această abordare facilitează nu doar reacția rapidă la noi amenințări, ci și implicarea utilizatorilor în ciclul de îmbunătățire continuă a modelului, deschizând perspectiva dezvoltării unui sistem adaptiv, capabil să evolueze pe baza feedback-ului colectiv.

Mai mult, evaluarea extensiei nu s-a limitat doar la performanțele pe date istorice, ci a inclus și o componentă de testare robustă, cu rulări multiple pentru fiecare caz analizat, pentru a elimina variațiile contextuale și a asigura validitatea statistică a rezultatelor. Această abordare metodologică asigură că performanțele raportate nu sunt rezultatul întâmplător al unei serii limitate de teste, ci reflectă comportamentul real al sistemului în scenarii variate de utilizare.

În concluzie, extensia WebSentinel reprezintă un exemplu reușit de integrare a inteligenței artificiale în instrumente de securitate cibernetică pentru utilizatorul final, îmbinând cu succes **precizia ridicată a detecției (91.35%)**, **viteza de răspuns de 64 ms** și **flexibilitatea arhitecturală**. Aceste rezultate validează conceptul și deschid drumul către dezvoltări ulterioare, precum integrarea unui sistem de învățare continuă bazat pe feedback-ul utilizatorilor, extinderea capacității de detecție la alte tipuri de amenințări web și adaptarea modelului pentru platforme diverse. Astfel, soluția propusă nu doar răspunde cerințelor actuale, ci anticipează și evoluția peisajului amenințărilor cibernetică, oferind o bază solidă pentru protecția proactivă a utilizatorilor în mediul online.

7. Bibliografie

- [1] [Online]. Available: <https://safebrowsing.google.com/>. [Accessed 20 04 2025].
- [2] X.-S. Yang, Harmony Search as a Metaheuristic Algorithm, 2009.
- [3] P. C. Thaksen J. Parvat, A Novel Approach to Deep Packet Inspection for Intrusion Detection, 2015.
- [4] [Online]. Available: <https://towardsdatascience.com/understanding-random-forest-58381e0602d2>. [Accessed 27 11 2023].
- [5] [Online]. Available: <https://rpubs.com/Diptirtha99/677810>. [Accessed 27 11 2023].
- [6] [Online]. Available: <https://www.geeksforgeeks.org/k-nearest-neighbours/>. [Accessed 27 11 2023].
- [7] [Online]. Available: <https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm>. [Accessed 27 11 2023].
- [8] [Online]. Available: <https://www.spiceworks.com/tech/big-data/articles/what-is-support-vector-machine/>. [Accessed 27 11 2023].
- [9] [Online]. Available: <https://iq.opengenus.org/gaussian-naive-bayes/>. [Accessed 27 11 2023].
- [10] [Online]. Available: <https://www.geeksforgeeks.org/ml-xgboost-extreme-gradient-boosting/>. [Accessed 27 11 2023].
- [11] Somesha, M., Pais, A.R., Rao, R.S. et al. Efficient deep learning techniques for the detection of phishing websites. *Sādhana* 45, 165 (2020).
- [12] N. Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma and H. Fujita, "Deep Learning for Phishing Detection: Taxonomy, Current Challenges and Future Directions," in *IEEE Access*, vol. 10, pp. 36429-36463, 2022, doi: 10.1109/ACCESS.2022.3151903..
- [13] [Online]. Available: <https://www.kaggle.com/datasets/shashwatwork/phishing-dataset-for-machine-learning>. [Accessed 26 11 2023].
- [14] [Online]. Available: <https://towardsdatascience.com/phishing-domain-detection-with-ml-5be9c99293e5>. [Accessed 26 11 2023].

- [15] A. Abunadi, O. Akanbi and A. Zainal, "Feature extraction process: A phishing detection approach," 2013 13th International Conference on Intelligent Systems Design and Applications, Salangor, Malaysia.
- [16] "S. Ali, M. Shahbaz and K. Jamil, "Entropy-Based Feature Selection Classification Approach for Detecting Phishing Websites," 2019 13th International Conference on Open Source Systems and Technologies (ICOSST), Lahore, Pakistan, 2019".
- [17] [Online]. Available: <https://www.tensorflow.org/js>. [Accessed 12 03 2025].
- [18] [Online]. Available: <https://react.dev/learn>. [Accessed 16 03 2025].
- [19] [Online]. Available: <https://www.simplilearn.com/tutorials/reactjs-tutorial/what-is-reactjs..> [Accessed 09 03 2025].
- [20] [Online]. Available: <https://github.com/ancamariac/chrome-extension/tree/main/web-sentinel>. [Accessed 24 05 2025].
- [21] "virustotal," [Online]. Available: <https://docs.virustotal.com/docs/how-it-works>. [Accessed 25 05 2025].
- [22] R. P. N. R. J. V. G. D. A. A. R. K. M. J. Q. T. R. A. G.-B. D. H. C. C. D. K. R. H. James Kirkpatrick, Overcoming catastrophic forgetting in neural networks.
- [23] [Online]. Available: <https://medium.com/biased-algorithms/how-to-freeze-layers-in-tensorflow-c1b5c98f2158>. [Accessed 10 03 2025].